

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOSROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 1
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 7, 2026
Experiment / Trial No.	Trial 1

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	100	Kept at the notebook default of 100x100.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Left as grayscale to keep the input tensor small while the architecture and training schedule were still being tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly

				through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Raised from default 0.1 to 0.2 so the held-back validation slice is large enough (roughly a fifth of the training images) to give a stable, less noisy accuracy estimate each epoch, at only a modest cost in training data.

3. Model Architecture Hyperparameters

Trial 1

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	16	Started every conv layer at the default of 16 filters as the baseline reading for this dataset/pipeline before any capacity was added.	49.90%	49.44%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	16	Started every conv layer at the default of 16 filters as the baseline reading for this dataset/pipeline before any capacity was added.		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	16	Set to 16, continuing the doubling pattern (fewer filters in early layers		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	16	Reduced from the notebook default of 64 down to 16 for this very first run, to keep the whole network small and fast to iterate on while the rest of the pipeline was still being shaken out.		
Dropout Rate	0.0 - 0.5	None (not used)	none	No dropout was used yet in this early trial.		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid was used		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	49.90%	49.44%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.001	Left at Adam's default (0.001) for the first run.		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Batch Size	8 / 16 / 32 / 64 / 128	32	128	Raised from the default of 32 to 128 to speed up each epoch on CPU by processing more images per gradient step, at the cost of a somewhat noisier gradient estimate.		
Number of Epochs	e.g. 10 - 200	200	5	Limited to 5 epochs purely to keep the first CPU-only training run fast while the pipeline was still being debugged.		
Early Stopping	Yes (patience = ___) / No	No	No	Not used		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	49.90%	90%
Validation Accuracy	49.44%	90%
Test Accuracy	Not Measured	
Test Loss	Not Measured	
Precision / Recall / F1 (macro avg)	Not Measured	

5. a. Paste here all screenshots/figures of the tuned model

No screenshot evidence was captured for this early trial; results were read directly from the notebook output at the time.

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Baseline run to confirm the data pipeline and training loop work end-to-end before any real tuning: minimal filters, small dense layer, only 5 epochs, notebook-default learning rate. Training accuracy 49.9% and validation accuracy 49.4% came out essentially at chance level for a binary task, so this became the floor everything else is judged against.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 2
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 7, 2026
Experiment / Trial No.	Trial 2

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	100	Kept at the notebook default of 100x100.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Left as grayscale to keep the input tensor small while the architecture and training schedule were still being tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 2

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	32	Raised every conv layer's filter count uniformly to 32 to test whether more raw capacity by itself would move accuracy off the trial-1 baseline, while epochs (5) and batch size were left similar.	49.62%	49.44%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	32	Raised every conv layer's filter count uniformly to 32 to test whether more raw capacity by itself would move accuracy off the trial-1 baseline, while epochs (5) and batch size were left similar.		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	32	Set to 32, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64, the notebook default.		
Dropout Rate	0.0 - 0.5	None (not used)	none	No dropout was used yet in this early trial.		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	49.62%	49.44%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of this size than plain SGD, without needing separate		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.001	Kept at the Adam default of 0.001.		
Batch Size	8 / 16 / 32 / 64 / 128	32	32	Reduced from 128 to 32.		
Number of Epochs	e.g. 10 - 200	200	5	Kept at 5 epochs, unchanged from the previous trial.		
Early Stopping	Yes (patience = ___) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the full accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an EarlyStopping(patience=...) callback is a natural next step once a longer, GPU-accelerated run is affordable.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	49.62%	90%
Validation Accuracy	49.44%	90%
Test Accuracy	Not computed	
Test Loss	Not computed	
Precision / Recall / F1 (macro avg)	Not computed	

5. a. Paste here all screenshots/figures of the tuned model

No screenshot evidence was captured for this early trial; results were read directly from the notebook output at the time.

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Filters raised uniformly to 32 across all conv layers to test whether raw capacity alone would move accuracy off the trial-1 baseline, with epochs and batch size otherwise similar. Result (49.6% / 49.4%) was unchanged from trial 1 - pointing to the short 5-epoch budget, not filter count, as the real bottleneck at this stage.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 3
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 8, 2026
Experiment / Trial No.	Trial 3

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	100	Kept at the notebook default of 100x100.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Left as grayscale to keep the input tensor small while the architecture and training schedule were still being tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 3

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	64	Raised every conv layer's filter count uniformly to 64 to test whether more raw capacity by itself would move accuracy off the trial-1 baseline, while epochs (5) and batch size were left similar.	49.54%	49.44%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	64	Set to 64 — see the Conv2D Layer 1 justification for the reasoning behind this trial's filter progression across the stack.		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	64	Set to 64, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64, the notebook default.		
Dropout Rate	0.0 - 0.5	None (not used)	none	No dropout was used yet in this early trial.		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	49.54%	49.44%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.001	Kept at the Adam default of 0.001.		
Batch Size	8 / 16 / 32 / 64 / 128	32	32	Kept at 32, unchanged from the previous trial, since it was already giving stable training and wasn't the focus of this round of tuning.		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Number of Epochs	e.g. 10 - 200	200	5	Kept at 5 epochs, unchanged from the previous trial.		
Early Stopping	Yes (patience = ____) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the full accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an EarlyStopping(patience=...) callback is a natural next step once a longer, GPU-accelerated run is affordable.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	49.54%	90%
Validation Accuracy	49.44%	90%
Test Accuracy	Not Computed	
Test Loss	Not Computed	
Precision / Recall / F1 (macro avg)	Not Computed	

5. a. Paste here all screenshots/figures of the tuned model

No screenshot evidence was captured for this early trial; results were read directly from the notebook output at the time.

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Filters raised uniformly to 64 across all conv layers to test whether raw capacity alone would move accuracy off the trial-1 baseline, with epochs and batch size otherwise similar. Result (49.5% / 49.4%) was unchanged from trial 1 - pointing to the short 5-epoch budget, not filter count, as the real bottleneck at this stage.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in Loading_Dataset_and_CNN.ipynb for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 4
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 8, 2026
Experiment / Trial No.	Trial 4

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	100	Kept default of 100x100.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Left as grayscale to keep the input tensor small (one channel instead of three) while the architecture and training schedule were still being tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 4

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	16	Filters set to [16, 64, 64] across the conv stack (previous trial: [64, 64, 64]) — capacity was reduced in layer(s) 1 to see the effect of deeper-layer capacity versus first-layer capacity on accuracy.	83.19%	82.58%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	64	Set to 64 — Filters set to [16, 64, 64] across the conv stack (previous trial: [64, 64, 64]) — capacity was reduced in layer(s) 1 to see the effect of deeper-layer capacity versus first-layer capacity on accuracy.		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	64	Set to 64, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64 default.		
Dropout Rate	0.0 - 0.5	None (not used)	none	No dropout was used yet in this early trial.		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	83.19%	82.58%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.001	Kept at the Adam default of 0.001.		
Batch Size	8 / 16 / 32 / 64 / 128	32	128	Raised from 32 to 128 to speed up training (fewer, larger steps per epoch) given the ~25,000-image dataset size, trading off a somewhat noisier gradient estimate for faster epochs.		
Number of Epochs	e.g. 10 - 200	200	5	Kept at 5 epochs, unchanged from the previous trial.		
Early Stopping	Yes (patience = ___) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the full accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an EarlyStopping(patience=...) callback is a natural next step once a longer, GPU-accelerated run is affordable.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	83.19%	90%

Metric	Value Obtained	Target / Acceptable Range
Validation Accuracy	82.58%	90%
Test Accuracy	Not computed	
Test Loss	Not computed	
Precision / Recall / F1 (macro avg)	Not computed	

5. a. Paste here all screenshots/figures of the tuned model

4. Training Hyperparameters (Batch size, number of epochs) 2. Data Preprocessing Parameters (validation split)

```
[73]: history = model.fit(X, y, batch_size=128, epochs=5, validation_split=0.2)

Epoch 1/5
157/157 ————— 26s 150ms/step - accuracy: 0.6209 - loss: 0.6421 - val_accuracy: 0.6670 - val_loss: 0.6173
Epoch 2/5
157/157 ————— 23s 143ms/step - accuracy: 0.7563 - loss: 0.5036 - val_accuracy: 0.7478 - val_loss: 0.5131
Epoch 3/5
157/157 ————— 22s 143ms/step - accuracy: 0.7918 - loss: 0.4460 - val_accuracy: 0.8034 - val_loss: 0.4384
Epoch 4/5
157/157 ————— 23s 144ms/step - accuracy: 0.8095 - loss: 0.4140 - val_accuracy: 0.8108 - val_loss: 0.4150
Epoch 5/5
157/157 ————— 23s 144ms/step - accuracy: 0.8319 - loss: 0.3778 - val_accuracy: 0.8258 - val_loss: 0.3984
```

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: filters changed to [16, 64, 64]; batch size changed to 128. Validation accuracy improved to 82.6% (from 49.4%), with training accuracy at 83.2%.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in Loading_Dataset_and_CNN.ipynb for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 5
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 9, 2026
Experiment / Trial No.	Trial 5

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	100	Kept at the default of 100x100.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Left as grayscale to keep the input tensor small while the architecture and training schedule were still being tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 — Filters	16 / 32 / 64 / 128	16	16	Filters set to [16, 32, 64] across the conv stack (previous trial: [16, 64, 64]) — capacity was reduced in layer(s) 2 to see the effect of deeper-	80.35%	79.14%

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				layer capacity versus first-layer capacity on accuracy.		
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	32	Set to 32, capacity was reduced in layer(s) 2 to see the effect of deeper-layer capacity versus first-layer capacity on		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	64	Set to 64, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64 default.		
Dropout Rate	0.0 - 0.5	None (not used)	none	No dropout was used yet in this early trial.		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	80.35%	79.14%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.001	Kept at the Adam default of 0.001.		
Batch Size	8 / 16 / 32 / 64 / 128	32	128	Kept at 128, unchanged from the previous trial, since it was already giving stable training and wasn't the focus of this round of tuning.		
Number of Epochs	e.g. 10 - 200	200	5	Kept at 5 epochs, unchanged from the previous trial.		
Early Stopping	Yes (patience = ___) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the full accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an EarlyStopping(patience=...) callback is a natural next step once a longer, GPU-accelerated run is affordable.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	80.35%	90%
Validation Accuracy	79.14%	90%
Test Accuracy	Not computed	
Test Loss	Not computed	
Precision / Recall / F1 (macro avg)	Not computed	

5. a. Paste here all screenshots/figures of the tuned model

```

1]: history = model.fit(X, y, batch_size=128, epochs=5, validation_split=0.2)

Epoch 1/5
157/157 — 20s 105ms/step - accuracy: 0.6019 - loss: 0.6557 - val_accuracy: 0.6796 - val_loss: 0.5937
Epoch 2/5
157/157 — 16s 100ms/step - accuracy: 0.7096 - loss: 0.5619 - val_accuracy: 0.7474 - val_loss: 0.5116
Epoch 3/5
157/157 — 16s 100ms/step - accuracy: 0.7653 - loss: 0.4872 - val_accuracy: 0.7768 - val_loss: 0.4688
Epoch 4/5
157/157 — 15s 97ms/step - accuracy: 0.7885 - loss: 0.4537 - val_accuracy: 0.7840 - val_loss: 0.4636
Epoch 5/5
157/157 — 15s 98ms/step - accuracy: 0.8035 - loss: 0.4247 - val_accuracy: 0.7914 - val_loss: 0.4573

```

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: filters changed to [16, 32, 64]. Validation accuracy dropped to 79.1% (from 82.6%), with training accuracy at 80.3%.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in Loading_Dataset_and_CNN.ipynb for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 6
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 9, 2026
Experiment / Trial No.	Trial 6

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	100	Kept at the default of 100x100.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Left as grayscale to keep the input tensor small while the architecture and training schedule were still being tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 6

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
----------------	----------------------------	------------------	--------------	---------------	-------------------	---------------------

Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	32	Filters set to [32, 64, 64] across the conv stack (previous trial: [16, 32, 64]) — capacity was increased in layer(s) 1, 2 to see the effect of deeper-layer capacity versus first-layer capacity on accuracy.	82.76%	80.76%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	64	Set to 64 - capacity was increased in layer(s) 1, 2 to see the effect of deeper-layer capacity versus first-layer capacity on accuracy.		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	64	Set to 64, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64.		

Dropout Rate	0.0 - 0.5	None (not used)	none	No dropout was used yet in this early trial.		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	82.76%	80.76%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.001	Kept at the Adam default of 0.001.		
Batch Size	8 / 16 / 32 / 64 / 128	32	128	Kept at 128, unchanged from the previous trial, since it was already giving stable training and wasn't the focus of this round of tuning.		
Number of Epochs	e.g. 10 - 200	200	5	Kept at 5 epochs, unchanged from the previous trial.		
Early Stopping	Yes (patience = ____) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the full accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an EarlyStopping(patience=...) callback is a natural next step once a longer, GPU-accelerated run is affordable.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	82.76%	90%
Validation Accuracy	80.76%	90%
Test Accuracy	Not Measured	
Test Loss	Not Measured	
Precision / Recall / F1 (macro avg)	Not Measured	

5. a. Paste here all screenshots/figures of the tuned model

```
[16]: history = model.fit(X, y, batch_size=128, epochs=5, validation_split=0.2)

Epoch 1/5
157/157 — 38s 228ms/step - accuracy: 0.6008 - loss: 0.6541 - val_accuracy: 0.6644 - val_loss: 0.6150
Epoch 2/5
157/157 — 36s 227ms/step - accuracy: 0.7339 - loss: 0.5358 - val_accuracy: 0.7328 - val_loss: 0.5333
Epoch 3/5
157/157 — 36s 227ms/step - accuracy: 0.7707 - loss: 0.4806 - val_accuracy: 0.7754 - val_loss: 0.4709
Epoch 4/5
157/157 — 36s 226ms/step - accuracy: 0.8018 - loss: 0.4298 - val_accuracy: 0.7912 - val_loss: 0.4443
Epoch 5/5
157/157 — 36s 228ms/step - accuracy: 0.8276 - loss: 0.3828 - val_accuracy: 0.8076 - val_loss: 0.4167

[17]: model.summary()
```

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: filters changed to [32, 64, 64]. Validation accuracy improved to 80.8% (from 79.1%), with training accuracy at 82.8%.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 7
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 10, 2026
Experiment / Trial No.	Trial 7

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	100	Kept the default 100x100.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Left as grayscale to keep the input tensor small while the architecture and training schedule were still being tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly through many hyperparameter combinations.

Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.
------------------------------------	-----------	-----	-----	--

3. Model Architecture Hyperparameters

Trial 7

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	16	Filters set to [16, 128, 128] across the conv stack (previous trial: [32, 64, 64]) — capacity was increased in layer(s) 1, 2, 3 to see the effect of deeper-layer capacity versus first-layer capacity on accuracy.	82.55%	81.20%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	128	Set to 128, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	128	Set to 128, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		

Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64, the notebook default.		
Dropout Rate	0.0 - 0.5	None (not used)	none	No dropout was used yet in this early trial.		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	82.55%	81.20%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.001	Kept at the Adam default of 0.001.		
Batch Size	8 / 16 / 32 / 64 / 128	32	128	Kept at 128, unchanged from the previous trial, since it was already giving stable training and wasn't the focus of this round of tuning.		
Number of Epochs	e.g. 10 - 200	200	5	Kept at 5 epochs, unchanged from the previous trial.		
Early Stopping	Yes (patience = ____) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the full accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an EarlyStopping(patience=...) callback is a natural next step once a longer, GPU-accelerated run is affordable.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	82.55%	90%
Validation Accuracy	81.20%	90%
Test Accuracy	Not Measured	
Test Loss	Not Measured	
Precision / Recall / F1 (macro avg)	Not Measured	

5. a. Paste here all screenshots/figures of the tuned model

```
[16]: history = model.fit(X, y, batch_size=128, epochs=5, validation_split=0.2)

Epoch 1/5
157/157 — 53s 320ms/step - accuracy: 0.5987 - loss: 0.6529 - val_accuracy: 0.7028 - val_loss: 0.5717
Epoch 2/5
157/157 — 49s 315ms/step - accuracy: 0.7350 - loss: 0.5300 - val_accuracy: 0.7514 - val_loss: 0.5045
Epoch 3/5
157/157 — 50s 315ms/step - accuracy: 0.7767 - loss: 0.4698 - val_accuracy: 0.7838 - val_loss: 0.4632
Epoch 4/5
157/157 — 49s 315ms/step - accuracy: 0.8071 - loss: 0.4175 - val_accuracy: 0.7932 - val_loss: 0.4380
Epoch 5/5
157/157 — 49s 314ms/step - accuracy: 0.8255 - loss: 0.3857 - val_accuracy: 0.8120 - val_loss: 0.4059

[17]: model.summary()
```

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: filters changed to [16, 128, 128]. Validation accuracy improved to 81.2% (from 80.8%), with training accuracy at 82.5%.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 8
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 10, 2026
Experiment / Trial No.	Trial 8

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	100	Kept at the notebook default of 100x100.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Left as grayscale to keep the input tensor small while the architecture and training schedule were still being tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 8

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	16	Filters set to [16, 64, 64] across the conv stack (previous trial: [16, 128, 128]) — capacity was reduced in layer(s) 2, 3 to see the effect of deeper-layer capacity versus first-layer capacity on accuracy.	91.74%	84.30%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D ; (2,2) / (3,3)	MaxPooling2D , (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	64	Set to 64, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D ; (2,2) / (3,3)	MaxPooling2D , (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	64	Set to 64, continuing the doubling pattern (fewer filters in early layers that detect		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D ; (2,2) / (3,3)	MaxPooling2D , (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64, the notebook default.		
Dropout Rate	0.0 - 0.5	None (not used)	none	No dropout was used yet in this early trial.		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	91.74%	84.30%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.001	Kept at the Adam default of 0.001.		
Batch Size	8 / 16 / 32 / 64 / 128	32	128	Kept at 128, unchanged from the previous trial, since it was already giving stable training and wasn't the focus of this round of tuning.		
Number of Epochs	e.g. 10 - 200	200	10	Increased from 5 to 10 epochs because accuracy in the previous trial was still climbing when training stopped, and training is CPU-only (no native GPU support on Windows for TF >=2.11) so each extra epoch is expensive.		
Early Stopping	Yes (patience = ____) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				slow), the plan was to read the full accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an EarlyStopping(patience=...) callback is a natural next step once a longer, GPU-accelerated run is affordable.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	91.74%	90%
Validation Accuracy	84.30%	90%
Test Accuracy	Not Measured	
Test Loss	Not Measured	
Precision / Recall / F1 (macro avg)	Not Measured	

5. a. Paste here all screenshots/figures of the tuned model

```
[16]: history = model.fit(X, y, batch_size=128, epochs=10, validation_split=0.2)
```

Epoch 1/10
157/157 ————— 26s 148ms/step - accuracy: 0.6316 - loss: 0.6349 - val_accuracy: 0.7246 - val_loss: 0.5452
Epoch 2/10
157/157 ————— 23s 143ms/step - accuracy: 0.7479 - loss: 0.5116 - val_accuracy: 0.7954 - val_loss: 0.4410
Epoch 3/10
157/157 ————— 23s 143ms/step - accuracy: 0.7839 - loss: 0.4582 - val_accuracy: 0.8084 - val_loss: 0.4227
Epoch 4/10
157/157 ————— 23s 143ms/step - accuracy: 0.8104 - loss: 0.4096 - val_accuracy: 0.8202 - val_loss: 0.4015
Epoch 5/10
157/157 ————— 23s 145ms/step - accuracy: 0.8415 - loss: 0.3595 - val_accuracy: 0.7732 - val_loss: 0.5090
Epoch 6/10
157/157 ————— 23s 144ms/step - accuracy: 0.8579 - loss: 0.3243 - val_accuracy: 0.8274 - val_loss: 0.3807
Epoch 7/10
157/157 ————— 22s 143ms/step - accuracy: 0.8765 - loss: 0.2820 - val_accuracy: 0.8306 - val_loss: 0.3888
Epoch 8/10
157/157 ————— 23s 144ms/step - accuracy: 0.8965 - loss: 0.2482 - val_accuracy: 0.8372 - val_loss: 0.4003
Epoch 9/10
157/157 ————— 23s 145ms/step - accuracy: 0.9024 - loss: 0.2283 - val_accuracy: 0.8444 - val_loss: 0.4139
Epoch 10/10
157/157 ————— 24s 150ms/step - accuracy: 0.9174 - loss: 0.1993 - val_accuracy: 0.8430 - val_loss: 0.4003

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: filters changed to [16, 64, 64]; epochs changed to 10. Validation accuracy improved to 84.3% (from 81.2%), with training accuracy at 91.7%.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in Loading_Dataset_and_CNN.ipynb for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 9
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 10, 2026
Experiment / Trial No.	Trial 9

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	100	Kept at the notebook default of 100x100.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Left as grayscale to keep the input tensor small while the architecture and training schedule were still being tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
				random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 9

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	32	Filters set to [32, 64, 64] across the conv stack (previous trial: [16, 64, 64]) — capacity was increased in layer(s) 1 to see the effect of deeper-layer capacity versus first-layer capacity on accuracy.	90.72%	84.30%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D ; (2,2) / (3,3)	MaxPooling2D , (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	64	Set to 64 — see the Conv2D Layer 1 justification for the reasoning behind this trial's filter progression across the stack.		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D ; (2,2) / (3,3)	MaxPooling2D , (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations,		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	64	Set to 64, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D ; (2,2) / (3,3)	MaxPooling2D , (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64, the notebook default.		
Dropout Rate	0.0 - 0.5	None (not used)	none	No dropout was used yet in this early trial.		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	90.72%	84.30%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.001	Kept at the Adam default of 0.001.		
Batch Size	8 / 16 / 32 / 64 / 128	32	128	Kept at 128, unchanged from the previous trial, since it was already giving stable training and wasn't the focus of this round of tuning.		
Number of Epochs	e.g. 10 - 200	200	10	Kept at 10 epochs, unchanged from the previous trial.		
Early Stopping	Yes (patience = ___) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the full		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an EarlyStopping(patience=...) callback is a natural next step once a longer, GPU-accelerated run is affordable.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	90.72%	90%
Validation Accuracy	84.30%	90%
Test Accuracy	Not Measured	
Test Loss	Not Measured	
Precision / Recall / F1 (macro avg)	Not Measured	

5. a. Paste here all screenshots/figures of the tuned model

```
[16]: history = model.fit(X, y, batch_size=128, epochs=10, validation_split=0.2)

Epoch 1/10
157/157 — 38s 230ms/step - accuracy: 0.5584 - loss: 0.6824 - val_accuracy: 0.6212 - val_loss: 0.6478
Epoch 2/10
157/157 — 36s 227ms/step - accuracy: 0.6812 - loss: 0.5938 - val_accuracy: 0.7230 - val_loss: 0.5524
Epoch 3/10
157/157 — 36s 226ms/step - accuracy: 0.7544 - loss: 0.5029 - val_accuracy: 0.7646 - val_loss: 0.4827
Epoch 4/10
157/157 — 35s 226ms/step - accuracy: 0.7866 - loss: 0.4527 - val_accuracy: 0.7956 - val_loss: 0.4447
Epoch 5/10
157/157 — 36s 226ms/step - accuracy: 0.8168 - loss: 0.4011 - val_accuracy: 0.8160 - val_loss: 0.4134
Epoch 6/10
157/157 — 36s 227ms/step - accuracy: 0.8389 - loss: 0.3632 - val_accuracy: 0.8186 - val_loss: 0.4218
Epoch 7/10
157/157 — 36s 227ms/step - accuracy: 0.8553 - loss: 0.3288 - val_accuracy: 0.8318 - val_loss: 0.3896
Epoch 8/10
157/157 — 36s 226ms/step - accuracy: 0.8761 - loss: 0.2889 - val_accuracy: 0.8334 - val_loss: 0.3774
Epoch 9/10
157/157 — 36s 226ms/step - accuracy: 0.8910 - loss: 0.2537 - val_accuracy: 0.8434 - val_loss: 0.3849
Epoch 10/10
157/157 — 36s 229ms/step - accuracy: 0.9072 - loss: 0.2255 - val_accuracy: 0.8430 - val_loss: 0.4337

[17]: model.summary()
```

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: filters changed to [32, 64, 64]. Validation accuracy held steady to 84.3% (from 84.3%), with training accuracy at 90.7%.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in Loading_Dataset_and_CNN.ipynb for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 10
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 10, 2026
Experiment / Trial No.	Trial 10

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	100	Kept at the notebook default of 100x100.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Left as grayscale to keep the input tensor small while the architecture and training schedule were still being tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
				random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 10

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	16	Filters set to [16, 64, 64] across the conv stack (previous trial: [32, 64, 64]) — capacity was reduced in layer(s) 1 to see the effect of deeper-layer capacity versus first-layer capacity on accuracy.	92.13%	82.70%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	64	Set to 64, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		

Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	64	Set to 64, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64, the notebook default.		
Dropout Rate	0.0 - 0.5	None (not used)	none	No dropout was used yet in this early trial.		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	92.13%	82.70%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.001	Kept at the Adam default of 0.001.		
Batch Size	8 / 16 / 32 / 64 / 128	32	128	Kept at 128, unchanged from the previous trial, since it was already giving stable training and wasn't the focus of this round of tuning.		
Number of Epochs	e.g. 10 - 200	200	15	Increased from 10 to 15 epochs because accuracy in the previous trial was still climbing when training stopped, and training is CPU-only (no native GPU support on Windows for TF		

				>=2.11) so each extra epoch is expensive.		
Early Stopping	Yes (patience = __) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the full accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an EarlyStopping(patience=...) callback is a natural next step once a longer, GPU-accelerated run is affordable.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	92.13%	90%
Validation Accuracy	82.70%	90%
Test Accuracy	Not Measured	
Test Loss	Not Measured	
Precision / Recall / F1 (macro avg)	Not Measured	

5. a. Paste here all screenshots/figures of the tuned model

```

: history = model.fit(X, y, batch_size=128, epochs=15, validation_split=0.2)
Epoch 1/15
157/157 — 25s 145ms/step - accuracy: 0.5568 - loss: 0.6880 - val_accuracy: 0.5944 - val_loss: 0.6579
Epoch 2/15
157/157 — 23s 147ms/step - accuracy: 0.6817 - loss: 0.5921 - val_accuracy: 0.7364 - val_loss: 0.5278
Epoch 3/15
157/157 — 22s 142ms/step - accuracy: 0.7510 - loss: 0.5068 - val_accuracy: 0.7626 - val_loss: 0.4880
Epoch 4/15
157/157 — 22s 143ms/step - accuracy: 0.7771 - loss: 0.4702 - val_accuracy: 0.7760 - val_loss: 0.4675
Epoch 5/15
157/157 — 23s 144ms/step - accuracy: 0.7989 - loss: 0.4366 - val_accuracy: 0.7932 - val_loss: 0.4427
Epoch 6/15
157/157 — 22s 143ms/step - accuracy: 0.8148 - loss: 0.4058 - val_accuracy: 0.8050 - val_loss: 0.4226
Epoch 7/15
157/157 — 22s 142ms/step - accuracy: 0.8337 - loss: 0.3724 - val_accuracy: 0.8206 - val_loss: 0.4021
Epoch 8/15
157/157 — 22s 141ms/step - accuracy: 0.8478 - loss: 0.3486 - val_accuracy: 0.8000 - val_loss: 0.4405
Epoch 9/15
157/157 — 23s 143ms/step - accuracy: 0.8586 - loss: 0.3237 - val_accuracy: 0.8188 - val_loss: 0.4281
Epoch 10/15
157/157 — 22s 143ms/step - accuracy: 0.8717 - loss: 0.3006 - val_accuracy: 0.8308 - val_loss: 0.4055
Epoch 11/15
157/157 — 23s 145ms/step - accuracy: 0.8798 - loss: 0.2800 - val_accuracy: 0.8310 - val_loss: 0.4015
Epoch 12/15
157/157 — 23s 144ms/step - accuracy: 0.8921 - loss: 0.2543 - val_accuracy: 0.8128 - val_loss: 0.4492
Epoch 13/15
157/157 — 22s 143ms/step - accuracy: 0.9024 - loss: 0.2339 - val_accuracy: 0.8302 - val_loss: 0.4282
Epoch 14/15
157/157 — 23s 145ms/step - accuracy: 0.9136 - loss: 0.2100 - val_accuracy: 0.8236 - val_loss: 0.4805
Epoch 15/15
157/157 — 22s 143ms/step - accuracy: 0.9213 - loss: 0.1902 - val_accuracy: 0.8270 - val_loss: 0.4990

```

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: filters changed to [16, 64, 64]; epochs changed to 15. Validation accuracy dropped to 82.7% (from 84.3%), with training accuracy at 92.1%. The 9.4% gap between training and validation accuracy suggests some overfitting.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 11
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 11, 2026
Experiment / Trial No.	Trial 11

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	100	I used less than 100 but blurred.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Left as grayscale to keep the input tensor small while the architecture and training schedule were still being tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 11

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	32	Filters set to [32, 128, 128] across the conv stack (previous trial: [16, 64, 64]) — capacity was increased in layer(s) 1, 2, 3 to see the effect of deeper-layer capacity versus first-layer capacity on accuracy.	93.73%	84.54%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	128	Set to 128, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	128	Set to 128, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64, the notebook default.		
Dropout Rate	0.0 - 0.5	None (not used)	none	No dropout was used yet in this early trial.		

Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		
--------------------------------	--	---------	---------	------------------------	--	--

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	93.73%	84.54%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.001	Kept at the Adam default of 0.001.		
Batch Size	8 / 16 / 32 / 64 / 128	32	128	Kept at 128, unchanged from the previous trial, since it was already giving stable training and wasn't the focus of this round of tuning.		
Number of Epochs	e.g. 10 - 200	200	10	Pulled back from 15 to 10 epochs after the previous trial's training accuracy pulled noticeably ahead of validation accuracy in the later epochs, a sign that continuing to train that long was mostly overfitting rather than helping generalization.		
Early Stopping	Yes (patience = ___) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the full accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an EarlyStopping(patience=...) callback is a natural next step once a longer, GPU-accelerated run is affordable.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	93.73%	90%
Validation Accuracy	84.54%	90%
Test Accuracy	Not Measured	
Test Loss	Not Measured	
Precision / Recall / F1 (macro avg)	Not Measured	

5. a. Paste here all screenshots/figures of the tuned model

```
history = model.fit(X, y, batch_size=128, epochs=10, validation_split=0.2)
```

```
Epoch 1/10
157/157 — 75s 458ms/step - accuracy: 0.6253 - loss: 0.6377 - val_accuracy: 0.7086 - val_loss: 0.5649
Epoch 2/10
157/157 — 70s 443ms/step - accuracy: 0.7516 - loss: 0.5049 - val_accuracy: 0.7784 - val_loss: 0.4563
Epoch 3/10
157/157 — 72s 459ms/step - accuracy: 0.7983 - loss: 0.4365 - val_accuracy: 0.7876 - val_loss: 0.4675
Epoch 4/10
157/157 — 71s 450ms/step - accuracy: 0.8269 - loss: 0.3838 - val_accuracy: 0.7942 - val_loss: 0.4579
Epoch 5/10
157/157 — 70s 445ms/step - accuracy: 0.8582 - loss: 0.3291 - val_accuracy: 0.8410 - val_loss: 0.3728
Epoch 6/10
157/157 — 69s 443ms/step - accuracy: 0.8769 - loss: 0.2907 - val_accuracy: 0.8360 - val_loss: 0.4014
Epoch 7/10
157/157 — 70s 443ms/step - accuracy: 0.8937 - loss: 0.2530 - val_accuracy: 0.8464 - val_loss: 0.3843
Epoch 8/10
157/157 — 71s 455ms/step - accuracy: 0.9120 - loss: 0.2162 - val_accuracy: 0.8154 - val_loss: 0.4653
Epoch 9/10
157/157 — 70s 446ms/step - accuracy: 0.9250 - loss: 0.1823 - val_accuracy: 0.8484 - val_loss: 0.4247
Epoch 10/10
157/157 — 70s 445ms/step - accuracy: 0.9373 - loss: 0.1561 - val_accuracy: 0.8454 - val_loss: 0.4680
```

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: filters changed to [32, 128, 128]; epochs changed to 10. Validation accuracy improved to 84.5% (from 82.7%), with training accuracy at 93.7%. The 9.2% gap between training and validation accuracy suggests some overfitting.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 12
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 11, 2026
Experiment / Trial No.	Trial 12

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	100	Kept at the notebook default of 100x100.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Left as grayscale to keep the input tensor small while the architecture and training schedule were still being tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 12

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	32	Filters kept at [32, 128, 128], unchanged from the previous trial, so that the effect of the hyperparameter actually being varied this round could be isolated.	92.46%	87.14%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	128	Set to 128 continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features)..		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	128	Set to 128, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64, the notebook default.		
Dropout Rate	0.0 - 0.5	None (not used)	0.3	Introduced dropout at 0.3 once earlier trials started showing training accuracy		

				noticeably ahead of validation accuracy (a classic overfitting signal).		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	92.46%	87.14%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.001	Kept at the Adam default of 0.001.		
Batch Size	8 / 16 / 32 / 64 / 128	32	128	Kept at 128, unchanged from the previous trial, since it was already giving stable training and wasn't the focus of this round of tuning.		
Number of Epochs	e.g. 10 - 200	200	15	Increased from 10 to 15 epochs because accuracy in the previous trial was still climbing when training stopped, and training is CPU-only (no native GPU support on Windows for TF >=2.11) so each extra epoch is expensive.		
Early Stopping	Yes (patience = ___) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the full accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an EarlyStopping(patience=...) callback is a natural next step once a longer, GPU-accelerated run is affordable.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	92.46%	90%
Validation Accuracy	87.14%	90%
Test Accuracy	Not Measured	
Test Loss	Not Measured	
Precision / Recall / F1 (macro avg)	Not Measured	

5. a. Paste here all screenshots/figures of the tuned model

```
[16]: history = model.fit(X, y, batch_size=128, epochs=15, validation_split=0.2)
```

Epoch 1/15	157/157	75s	462ms/step	- accuracy: 0.5399	- loss: 0.6889	- val_accuracy: 0.6120	- val_loss: 0.6639
Epoch 2/15	157/157	71s	454ms/step	- accuracy: 0.6566	- loss: 0.6170	- val_accuracy: 0.6820	- val_loss: 0.5917
Epoch 3/15	157/157	71s	455ms/step	- accuracy: 0.7288	- loss: 0.5406	- val_accuracy: 0.7398	- val_loss: 0.5224
Epoch 4/15	157/157	71s	455ms/step	- accuracy: 0.7741	- loss: 0.4838	- val_accuracy: 0.8074	- val_loss: 0.4306
Epoch 5/15	157/157	72s	456ms/step	- accuracy: 0.8106	- loss: 0.4159	- val_accuracy: 0.8284	- val_loss: 0.3998
Epoch 6/15	157/157	71s	454ms/step	- accuracy: 0.8355	- loss: 0.3784	- val_accuracy: 0.8352	- val_loss: 0.3700
Epoch 7/15	157/157	72s	461ms/step	- accuracy: 0.8486	- loss: 0.3455	- val_accuracy: 0.8212	- val_loss: 0.4085
Epoch 8/15	157/157	72s	461ms/step	- accuracy: 0.8626	- loss: 0.3211	- val_accuracy: 0.8180	- val_loss: 0.3960
Epoch 9/15	157/157	73s	466ms/step	- accuracy: 0.8746	- loss: 0.2943	- val_accuracy: 0.8576	- val_loss: 0.3359
Epoch 10/15	157/157	71s	453ms/step	- accuracy: 0.8817	- loss: 0.2735	- val_accuracy: 0.8584	- val_loss: 0.3333
Epoch 11/15	157/157	72s	456ms/step	- accuracy: 0.8972	- loss: 0.2437	- val_accuracy: 0.8538	- val_loss: 0.3418
Epoch 12/15	157/157	71s	453ms/step	- accuracy: 0.9028	- loss: 0.2335	- val_accuracy: 0.8538	- val_loss: 0.3475
Epoch 13/15	157/157	71s	451ms/step	- accuracy: 0.9119	- loss: 0.2129	- val_accuracy: 0.8700	- val_loss: 0.3292
Epoch 14/15	157/157	71s	454ms/step	- accuracy: 0.9189	- loss: 0.1921	- val_accuracy: 0.8582	- val_loss: 0.3509
Epoch 15/15	157/157	71s	452ms/step	- accuracy: 0.9246	- loss: 0.1814	- val_accuracy: 0.8714	- val_loss: 0.3516

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: epochs changed to 15; dropout changed to 0.3. Validation accuracy improved to 87.1% (from 84.5%), with training accuracy at 92.5%.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 13
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 11, 2026
Experiment / Trial No.	Trial 13

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	100	Kept at the notebook default of 100x100.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Left as grayscale to keep the input tensor small while the architecture and training schedule were still being tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 13

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	32	Filters kept at [32, 128, 128], unchanged from the previous trial, so that the effect of the hyperparameter actually being varied this round could be isolated.	96.59%	86.50%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	128	Set to 128, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	128	Set to 128, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64, the notebook default.		
Dropout Rate	0.0 - 0.5	None (not used)	0.3	Kept at 0.3, which had already narrowed the train/validation gap in earlier trials without holding back training accuracy as		

				much as the 0.5 test did, so it was carried forward as the standard setting.		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	96.59%	86.50%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.001	Kept at the Adam default of 0.001.		
Batch Size	8 / 16 / 32 / 64 / 128	32	128	Kept at 128, unchanged from the previous trial, since it was already giving stable training and wasn't the focus of this round of tuning.		
Number of Epochs	e.g. 10 - 200	200	25	Increased from 15 to 25 epochs because accuracy in the previous trial was still climbing when training stopped, and training is CPU-only (no native GPU support on Windows for TF >=2.11) so each extra epoch is expensive.		
Early Stopping	Yes (patience = ___) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the full accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an EarlyStopping(patience=...) callback is a natural next step once a longer, GPU-accelerated run is affordable.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	96.59%	90%
Validation Accuracy	86.50%	90%
Test Accuracy	Not Measured	
Test Loss	Not Measured	
Precision / Recall / F1 (macro avg)	Not Measured	

5. a. Paste here all screenshots/figures of the tuned model

```
[16]: history = model.fit(X, y, batch_size=128, epochs=25, validation_split=0.2)
```

Epoch	Time	Step	Accuracy	Loss	Val Accuracy	Val Loss
Epoch 1/25	157/157	74s 458ms/step	0.6020	0.6584	0.6958	0.5873
Epoch 2/25	157/157	71s 450ms/step	0.7224	0.5476	0.7586	0.4935
Epoch 3/25	157/157	71s 453ms/step	0.7685	0.4802	0.7906	0.4355
Epoch 4/25	157/157	71s 454ms/step	0.7994	0.4328	0.8122	0.4097
Epoch 5/25	157/157	71s 453ms/step	0.8256	0.3893	0.8374	0.3723
Epoch 6/25	157/157	72s 456ms/step	0.8427	0.3537	0.8012	0.4147
Epoch 7/25	157/157	71s 450ms/step	0.8554	0.3310	0.8520	0.3449
Epoch 8/25	157/157	71s 453ms/step	0.8702	0.3035	0.8600	0.3247
Epoch 9/25	157/157	71s 451ms/step	0.8830	0.2766	0.8584	0.3308
Epoch 10/25	157/157	71s 451ms/step	0.8927	0.2580	0.8604	0.3172
Epoch 11/25	157/157	71s 452ms/step	0.8950	0.2465	0.8580	0.3262
Epoch 12/25	157/157	71s 450ms/step	0.9091	0.2181	0.8680	0.3239
Epoch 13/25	157/157	71s 450ms/step	0.9139	0.2099	0.8688	0.3331
Epoch 14/25	157/157	72s 458ms/step	0.9225	0.1875	0.8722	0.3221
Epoch 15/25	157/157	70s 449ms/step	0.9276	0.1770	0.8548	0.3686
Epoch 16/25	157/157	71s 451ms/step	0.9319	0.1644	0.8602	0.3516
Epoch 17/25	157/157	71s 453ms/step	0.9397	0.1529	0.8694	0.3481
Epoch 18/25	157/157	71s 451ms/step	0.9441	0.1383	0.8702	0.3721
Epoch 19/25	157/157	72s 457ms/step	0.9481	0.1296	0.8686	0.3558
Epoch 20/25	157/157	71s 454ms/step	0.9498	0.1238	0.8706	0.3643
Epoch 21/25	157/157	71s 450ms/step	0.9548	0.1150	0.8648	0.3970
Epoch 22/25	157/157	71s 454ms/step	0.9589	0.1065	0.8722	0.3804
Epoch 23/25	157/157	71s 450ms/step	0.9554	0.1138	0.8716	0.3936
Epoch 24/25	157/157	72s 456ms/step	0.9619	0.0964	0.8690	0.4091
Epoch 25/25	157/157	71s 451ms/step	0.9659	0.0888	0.8650	0.5123

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: epochs changed to 25. Validation accuracy dropped to 86.5% (from 87.1%), with training accuracy at 96.6%. The 10.1% gap between training and validation accuracy suggests some overfitting.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in Loading_Dataset_and_CNN.ipynb for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 14
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 13, 2026
Experiment / Trial No.	Trial 14

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	100	I used less than 100 but blurred.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Left as grayscale to keep the input tensor small while the architecture and training schedule were still being tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a random mix of cat and dog images instead of a long run of

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
				one class followed by the other, which would otherwise bias per-batch gradient estimates.
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 14

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	16	Filters set to [16, 64, 64] across the conv stack (previous trial: [32, 128, 128]) — capacity was reduced in layer(s) 1, 2, 3 to see the effect of deeper-layer capacity versus first-layer capacity on accuracy.	91.30%	84.86%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	64	Set to 64 — see the Conv2D Layer 1 justification for the reasoning behind this trial's filter progression across the stack.		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		

Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	64	Set to 64, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64, the notebook default.
Dropout Rate	0.0 - 0.5	None (not used)	0.3	Kept at 0.3, which had already narrowed the train/validation gap in earlier trials without holding back training accuracy as much as the 0.5 test did, so it was carried forward as the standard setting.
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	91.30%	84.86%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.001	Kept at the Adam default of 0.001.		
Batch Size	8 / 16 / 32 / 64 / 128	32	128	Kept at 128, unchanged from the previous trial, since it was already giving stable training and wasn't the focus of this round of tuning.		
Number of Epochs	e.g. 10 - 200	200	15	Pulled back from 25 to 15 epochs after the previous trial's training accuracy pulled noticeably ahead of validation accuracy in the later epochs, a sign that continuing to train that long was mostly overfitting rather than helping generalization.		
Early Stopping	Yes (patience = ____) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the		

				full accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an EarlyStopping(patience=...) callback is a natural next step once a longer, GPU-accelerated run is affordable.		
--	--	--	--	--	--	--

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	91.30%	
Validation Accuracy	84.86%	
Test Accuracy	Not Measured	
Test Loss	Not Measured	
Precision / Recall / F1 (macro avg)	Not Measured	

5. a. Paste here all screenshots/figures of the tuned model

```

history = model.fit(X, y, batch_size=128, epochs=15, validation_split=0.2)

Epoch 1/15
157/157 — 28s 158ms/step - accuracy: 0.6211 - loss: 0.6365 - val_accuracy: 0.7146 - val_loss: 0.5642
Epoch 2/15
157/157 — 24s 151ms/step - accuracy: 0.7311 - loss: 0.5369 - val_accuracy: 0.7122 - val_loss: 0.5565
Epoch 3/15
157/157 — 24s 151ms/step - accuracy: 0.7641 - loss: 0.4864 - val_accuracy: 0.7968 - val_loss: 0.4523
Epoch 4/15
157/157 — 24s 150ms/step - accuracy: 0.7939 - loss: 0.4449 - val_accuracy: 0.7834 - val_loss: 0.4641
Epoch 5/15
157/157 — 24s 151ms/step - accuracy: 0.8115 - loss: 0.4128 - val_accuracy: 0.8234 - val_loss: 0.4001
Epoch 6/15
157/157 — 24s 151ms/step - accuracy: 0.8352 - loss: 0.3753 - val_accuracy: 0.8290 - val_loss: 0.3852
Epoch 7/15
157/157 — 24s 152ms/step - accuracy: 0.8407 - loss: 0.3547 - val_accuracy: 0.8430 - val_loss: 0.3609
Epoch 8/15
157/157 — 24s 151ms/step - accuracy: 0.8560 - loss: 0.3299 - val_accuracy: 0.8410 - val_loss: 0.3611
Epoch 9/15
157/157 — 24s 151ms/step - accuracy: 0.8659 - loss: 0.3080 - val_accuracy: 0.8184 - val_loss: 0.4140
Epoch 10/15
157/157 — 24s 151ms/step - accuracy: 0.8808 - loss: 0.2794 - val_accuracy: 0.8526 - val_loss: 0.3565
Epoch 11/15
157/157 — 24s 152ms/step - accuracy: 0.8863 - loss: 0.2701 - val_accuracy: 0.8492 - val_loss: 0.3491
Epoch 12/15
157/157 — 24s 151ms/step - accuracy: 0.8911 - loss: 0.2511 - val_accuracy: 0.8534 - val_loss: 0.3489
Epoch 13/15
157/157 — 24s 150ms/step - accuracy: 0.8956 - loss: 0.2443 - val_accuracy: 0.8584 - val_loss: 0.3481
Epoch 14/15
157/157 — 24s 150ms/step - accuracy: 0.9063 - loss: 0.2253 - val_accuracy: 0.8540 - val_loss: 0.3549
Epoch 15/15
157/157 — 23s 149ms/step - accuracy: 0.9130 - loss: 0.2124 - val_accuracy: 0.8486 - val_loss: 0.3497

model.summary()

```

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: filters changed to [16, 64, 64]; epochs changed to 15. Validation accuracy dropped to 84.9% (from 86.5%), with training accuracy at 91.3%.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 15
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 13, 2026
Experiment / Trial No.	Trial 15

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	150	Pushed to the top of the range (150x150) to see whether more resolution alone would lift accuracy.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Left as grayscale to keep the input tensor small while the architecture and training schedule were still being tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set,

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
				so both the training and validation subsets contain a random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 15

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	16	Filters kept at [16, 64, 64], unchanged from the previous trial, so that the effect of the hyperparameter actually being varied this round could be isolated.	93.67%	84.72%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	64	Set to 64 — see the Conv2D Layer 1 justification for the reasoning behind this trial's filter progression across the stack.		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	64	Set to 64, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64, the notebook default.		
Dropout Rate	0.0 - 0.5	None (not used)	0.3	Kept at 0.3, which had already narrowed the train/validation gap in earlier trials without holding back training accuracy as much as the 0.5 test did, so it was carried forward as the standard setting.		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	93.67%	84.72%

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.001	Kept at the Adam default of 0.001.		
Batch Size	8 / 16 / 32 / 64 / 128	32	128	Kept at 128, unchanged from the previous trial, since it was already giving stable training and wasn't the focus of this round of tuning.		
Number of Epochs	e.g. 10 - 200	200	15	Kept at 15 epochs, unchanged from the previous trial.		
Early Stopping	Yes (patience = ___) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the full accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an EarlyStopping(patience=...) callback is a natural next step once a longer, GPU-accelerated run is affordable.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	93.67%	
Validation Accuracy	84.72%	
Test Accuracy	Not Measured	
Test Loss	Not Measured	
Precision / Recall / F1 (macro avg)	Not Measured	

5. a. Paste here all screenshots/figures of the tuned model

```

history = model.fit(X, y, batch_size=128, epochs=15, validation_split=0.2)

```

```

Epoch 1/15
157/157 ————— 67s 407ms/step - accuracy: 0.5903 - loss: 0.6694 - val_accuracy: 0.6660 - val_loss: 0.6142
Epoch 2/15
157/157 ————— 62s 392ms/step - accuracy: 0.6951 - loss: 0.5780 - val_accuracy: 0.6904 - val_loss: 0.5876
Epoch 3/15
157/157 ————— 62s 393ms/step - accuracy: 0.7514 - loss: 0.5099 - val_accuracy: 0.7740 - val_loss: 0.5025
Epoch 4/15
157/157 ————— 62s 394ms/step - accuracy: 0.7878 - loss: 0.4572 - val_accuracy: 0.8066 - val_loss: 0.4342
Epoch 5/15
157/157 ————— 61s 391ms/step - accuracy: 0.8157 - loss: 0.4099 - val_accuracy: 0.8192 - val_loss: 0.4110
Epoch 6/15
157/157 ————— 61s 390ms/step - accuracy: 0.8348 - loss: 0.3785 - val_accuracy: 0.8148 - val_loss: 0.4093
Epoch 7/15
157/157 ————— 63s 398ms/step - accuracy: 0.8488 - loss: 0.3488 - val_accuracy: 0.8160 - val_loss: 0.4068
Epoch 8/15
157/157 ————— 62s 394ms/step - accuracy: 0.8668 - loss: 0.3088 - val_accuracy: 0.8456 - val_loss: 0.3809
Epoch 9/15
157/157 ————— 65s 414ms/step - accuracy: 0.8819 - loss: 0.2788 - val_accuracy: 0.8436 - val_loss: 0.3745
Epoch 10/15
157/157 ————— 63s 400ms/step - accuracy: 0.8986 - loss: 0.2469 - val_accuracy: 0.8434 - val_loss: 0.3705
Epoch 11/15
157/157 ————— 62s 394ms/step - accuracy: 0.9042 - loss: 0.2328 - val_accuracy: 0.8522 - val_loss: 0.3790
Epoch 12/15
157/157 ————— 62s 394ms/step - accuracy: 0.9106 - loss: 0.2128 - val_accuracy: 0.8412 - val_loss: 0.4359
Epoch 13/15
157/157 ————— 62s 394ms/step - accuracy: 0.9179 - loss: 0.1987 - val_accuracy: 0.8486 - val_loss: 0.3964
Epoch 14/15
157/157 ————— 61s 389ms/step - accuracy: 0.9284 - loss: 0.1735 - val_accuracy: 0.8482 - val_loss: 0.4387
Epoch 15/15
157/157 ————— 61s 392ms/step - accuracy: 0.9367 - loss: 0.1554 - val_accuracy: 0.8472 - val_loss: 0.4472

```

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: image size changed to 150. Validation accuracy held steady to 84.7% (from 84.9%), with training accuracy at 93.7%. The 9.0% gap between training and validation accuracy suggests some overfitting.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 16
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 13, 2026
Experiment / Trial No.	Trial 16

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	128	I used less than 100 but blurred.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Left as grayscale to keep the input tensor small while the architecture and training schedule were still being tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.

Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 16

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	32	Filters set to [32, 32, 64, 64] across the conv stack (previous trial: [16, 64, 64]) — capacity was increased in layer(s) 1, 2 to see the effect of deeper-layer capacity versus first-layer capacity on accuracy.	92.85%	88.30%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	32	Set to 32 — see the Conv2D Layer 1 justification for the reasoning behind this trial's filter progression across the stack.		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	64	Set to 64, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		

Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 4 – Filters	16 / 32 / 64 / 128	16	64	Set to 64.		
Conv2D Layer 4 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 4 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 4 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64, the notebook default.		
Dropout Rate	0.0 - 0.5	None (not used)	0.3	Kept at 0.3, which had already narrowed the train/validation gap in earlier trials without holding back training accuracy as much as the 0.5 test did, so it was carried forward as the standard setting.		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	Standard choice for binary classification (Cat vs Dog).	92.85%	88.30%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam adapts the learning rate per-parameter automatically, which generally converges faster and more reliably than plain SGD without manual tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.001	Used Keras/Adam's default rate (didn't override it by passing a custom optimizer instance) since it's a well-tested starting point for image classification tasks.		
Batch Size	8 / 16 / 32 / 64 / 128	32	128	Larger batch size reduces training time per epoch and gave stable gradient estimates for this dataset size (~25,000 images).		
Number of Epochs	e.g. 10 - 200	200	15	Limited to 5 epochs due to CPU-only training time (no native GPU support on Windows for TF ≥2.11).		

Early Stopping	Yes (patience = ___) / No	No	No	Not used — training accuracy and validation accuracy were still both improving through epoch 5 with no sign of overfitting divergence, so stopping wasn't necessary within this short run.		
-----------------------	---------------------------	----	----	--	--	--

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	92.85%	
Validation Accuracy	88.30%	
Test Accuracy	Not Measured	
Test Loss	Not Measured	
Precision / Recall / F1 (macro avg)	Not Measured	

5. a. Paste here all screenshots/figures of the tuned model

```
[16]: history = model.fit(X, y, batch_size=128, epochs=15, validation_split=0.2)

Epoch 1/15
157/157 — 49s 287ms/step - accuracy: 0.6160 - loss: 0.6464 - val_accuracy: 0.7164 - val_loss: 0.5734
Epoch 2/15
157/157 — 43s 273ms/step - accuracy: 0.7286 - loss: 0.5490 - val_accuracy: 0.7696 - val_loss: 0.4820
Epoch 3/15
157/157 — 43s 275ms/step - accuracy: 0.7709 - loss: 0.4815 - val_accuracy: 0.7924 - val_loss: 0.4386
Epoch 4/15
157/157 — 43s 274ms/step - accuracy: 0.8032 - loss: 0.4261 - val_accuracy: 0.8114 - val_loss: 0.4098
Epoch 5/15
157/157 — 43s 276ms/step - accuracy: 0.8276 - loss: 0.3836 - val_accuracy: 0.8400 - val_loss: 0.3666
Epoch 6/15
157/157 — 43s 274ms/step - accuracy: 0.8504 - loss: 0.3444 - val_accuracy: 0.8590 - val_loss: 0.3307
Epoch 7/15
157/157 — 43s 273ms/step - accuracy: 0.8630 - loss: 0.3158 - val_accuracy: 0.8450 - val_loss: 0.3386
Epoch 8/15
157/157 — 43s 274ms/step - accuracy: 0.8708 - loss: 0.2950 - val_accuracy: 0.8750 - val_loss: 0.2962
Epoch 9/15
157/157 — 43s 274ms/step - accuracy: 0.8793 - loss: 0.2783 - val_accuracy: 0.8718 - val_loss: 0.3004
Epoch 10/15
157/157 — 43s 274ms/step - accuracy: 0.8902 - loss: 0.2544 - val_accuracy: 0.8728 - val_loss: 0.3037
Epoch 11/15
157/157 — 43s 274ms/step - accuracy: 0.9036 - loss: 0.2277 - val_accuracy: 0.8808 - val_loss: 0.2791
Epoch 12/15
157/157 — 43s 276ms/step - accuracy: 0.9043 - loss: 0.2250 - val_accuracy: 0.8750 - val_loss: 0.2994
Epoch 13/15
157/157 — 44s 278ms/step - accuracy: 0.9141 - loss: 0.2051 - val_accuracy: 0.8582 - val_loss: 0.3448
Epoch 14/15
157/157 — 43s 274ms/step - accuracy: 0.9218 - loss: 0.1901 - val_accuracy: 0.8670 - val_loss: 0.3112
Epoch 15/15
157/157 — 43s 274ms/step - accuracy: 0.9285 - loss: 0.1744 - val_accuracy: 0.8830 - val_loss: 0.2973

[*]: model.summary()
```

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: filters changed to [32, 32, 64, 64]; image size changed to 128. Validation accuracy improved to 88.3% (from 84.7%), with training accuracy at 92.8%.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 17
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 14, 2026
Experiment / Trial No.	Trial 17

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	128	I used less than 100 but blurred.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Left as grayscale to keep the input tensor small while the architecture and training schedule were still being tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.

Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 17

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	32	Filters kept at [32, 32, 64, 64], unchanged from the previous trial, so that the effect of the hyperparameter actually being varied this round could be isolated.	93.92%	87.22%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	32	Set to 32, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	64	Set to 64, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the		

				standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 4 – Filters	16 / 32 / 64 / 128	16	64	Set to 64.		
Conv2D Layer 4 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 4 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 4 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64, the notebook default.		
Dropout Rate	0.0 - 0.5	None (not used)	0.3	Kept at 0.3, which had already narrowed the train/validation gap in earlier trials without holding back training accuracy as much as the 0.5 test did, so it was carried forward as the standard setting.		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	93.92%	87.22%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.001	Kept at the Adam default of 0.001.		
Batch Size	8 / 16 / 32 / 64 / 128	32	64	Reduced from 128 to 64.		
Number of Epochs	e.g. 10 - 200	200	15	Kept at 15 epochs, unchanged from the previous trial.		
Early Stopping	Yes (patience = ___) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the full accuracy/loss curve for each fixed epoch budget by hand rather than let		

				the run stop automatically; adding an EarlyStopping(patience=...) callback is a natural next step once a longer, GPU-accelerated run is affordable.		
--	--	--	--	---	--	--

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	93.92%	
Validation Accuracy	87.22%	
Test Accuracy	Not Measured	
Test Loss	Not Measured	
Precision / Recall / F1 (macro avg)	Not Measured	

5. a. Paste here all screenshots/figures of the tuned model

```
history = model.fit(X, y, batch_size=64, epochs=15, validation_split=0.2)
```

Epoch 1/15
313/313 ————— 46s 140ms/step - accuracy: 0.5879 - loss: 0.6667 - val_accuracy: 0.6076 - val_loss: 0.6495
Epoch 2/15
313/313 ————— 43s 138ms/step - accuracy: 0.7168 - loss: 0.5546 - val_accuracy: 0.7582 - val_loss: 0.4974
Epoch 3/15
313/313 ————— 44s 141ms/step - accuracy: 0.7835 - loss: 0.4637 - val_accuracy: 0.7758 - val_loss: 0.4731
Epoch 4/15
313/313 ————— 46s 147ms/step - accuracy: 0.8139 - loss: 0.4042 - val_accuracy: 0.8094 - val_loss: 0.4117
Epoch 5/15
313/313 ————— 45s 144ms/step - accuracy: 0.8418 - loss: 0.3601 - val_accuracy: 0.8372 - val_loss: 0.3766
Epoch 6/15
313/313 ————— 45s 142ms/step - accuracy: 0.8540 - loss: 0.3376 - val_accuracy: 0.8378 - val_loss: 0.3519
Epoch 7/15
313/313 ————— 44s 142ms/step - accuracy: 0.8691 - loss: 0.3012 - val_accuracy: 0.8484 - val_loss: 0.3464
Epoch 8/15
313/313 ————— 49s 158ms/step - accuracy: 0.8815 - loss: 0.2767 - val_accuracy: 0.8518 - val_loss: 0.3345
Epoch 9/15
313/313 ————— 45s 144ms/step - accuracy: 0.8901 - loss: 0.2555 - val_accuracy: 0.8510 - val_loss: 0.3315
Epoch 10/15
313/313 ————— 45s 145ms/step - accuracy: 0.9021 - loss: 0.2326 - val_accuracy: 0.8672 - val_loss: 0.3139
Epoch 11/15
313/313 ————— 44s 142ms/step - accuracy: 0.9133 - loss: 0.2103 - val_accuracy: 0.8706 - val_loss: 0.3132
Epoch 12/15
313/313 ————— 45s 144ms/step - accuracy: 0.9216 - loss: 0.1913 - val_accuracy: 0.8670 - val_loss: 0.3140
Epoch 13/15
313/313 ————— 44s 142ms/step - accuracy: 0.9271 - loss: 0.1777 - val_accuracy: 0.8710 - val_loss: 0.3029
Epoch 14/15
313/313 ————— 45s 143ms/step - accuracy: 0.9324 - loss: 0.1666 - val_accuracy: 0.8566 - val_loss: 0.3711
Epoch 15/15
313/313 ————— 45s 144ms/step - accuracy: 0.9392 - loss: 0.1495 - val_accuracy: 0.8722 - val_loss: 0.3086

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: batch size changed to 64. Validation accuracy dropped to 87.2% (from 88.3%), with training accuracy at 93.9%.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 18
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 14, 2026
Experiment / Trial No.	Trial 18

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	128	Kept at 128x128, which had already shown a good accuracy/compute trade-off in earlier runs, so it was held fixed while other hyperparameters (color mode, batch size, epochs, learning rate) were the focus of this trial.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Left as grayscale to keep the input tensor small while the architecture and training schedule were still being tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.

Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 18

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	32	Filters kept at [32, 32, 64, 64], unchanged from the previous trial, so that the effect of the hyperparameter actually being varied this round could be isolated.	85.65%	85.64%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	32	Set to 32 continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	64	Set to 64, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		

Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 4 – Filters	16 / 32 / 64 / 128	16	64	Set to 64.		
Conv2D Layer 4 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 4 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 4 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	32	Reduced to 32 (from the usual 64) alongside a heavier dropout rate in this trial, to test a smaller, more regularized classifier head.		
Dropout Rate	0.0 - 0.5	None (not used)	0.5	Raised to 0.5 (from the usual 0.3) to test whether heavier regularization would close the train/validation gap seen in earlier trials.		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	85.65%	85.64%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.0001	Dropped to 0.0001 (10x lower than default) alongside the heavier-dropout, smaller-dense-layer test in this trial.		
Batch Size	8 / 16 / 32 / 64 / 128	32	32	Reduced from 64 to 32.		

Number of Epochs	e.g. 10 - 200	200	12	Pulled back from 15 to 12 epochs after the previous trial's training accuracy pulled noticeably ahead of validation accuracy in the later epochs, a sign that continuing to train that long was mostly overfitting rather than helping generalization.		
Early Stopping	Yes (patience = ___) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the full accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an EarlyStopping(patience=...) callback is a natural next step once a longer, GPU-accelerated run is affordable.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	85.65%	
Validation Accuracy	85.64%	
Test Accuracy	Not Measured	
Test Loss	Not Measured	
Precision / Recall / F1 (macro avg)	Not Measured	

5. a. Paste here all screenshots/figures of the tuned model

```
history = model.fit(X, y, batch_size=32, epochs=12, validation_split=0.2)
```

Epoch 1/12
625/625 — 48s 73ms/step - accuracy: 0.5730 - loss: 0.6707 - val_accuracy: 0.6680 - val_loss: 0.6115

Epoch 2/12
625/625 — 45s 72ms/step - accuracy: 0.6778 - loss: 0.6003 - val_accuracy: 0.7280 - val_loss: 0.5518

Epoch 3/12
625/625 — 45s 72ms/step - accuracy: 0.7186 - loss: 0.5536 - val_accuracy: 0.7612 - val_loss: 0.4987

Epoch 4/12
625/625 — 45s 72ms/step - accuracy: 0.7609 - loss: 0.5000 - val_accuracy: 0.7598 - val_loss: 0.4908

Epoch 5/12
625/625 — 45s 72ms/step - accuracy: 0.7806 - loss: 0.4678 - val_accuracy: 0.8140 - val_loss: 0.4331

Epoch 6/12
625/625 — 45s 72ms/step - accuracy: 0.7993 - loss: 0.4419 - val_accuracy: 0.8188 - val_loss: 0.4123

Epoch 7/12
625/625 — 45s 72ms/step - accuracy: 0.8114 - loss: 0.4195 - val_accuracy: 0.8298 - val_loss: 0.3938

Epoch 8/12
625/625 — 45s 72ms/step - accuracy: 0.8206 - loss: 0.4014 - val_accuracy: 0.8378 - val_loss: 0.3764

Epoch 9/12
625/625 — 45s 72ms/step - accuracy: 0.8318 - loss: 0.3817 - val_accuracy: 0.8462 - val_loss: 0.3620

Epoch 10/12
625/625 — 45s 73ms/step - accuracy: 0.8434 - loss: 0.3639 - val_accuracy: 0.8318 - val_loss: 0.3775

Epoch 11/12
625/625 — 45s 72ms/step - accuracy: 0.8450 - loss: 0.3569 - val_accuracy: 0.8426 - val_loss: 0.3601

Epoch 12/12
625/625 — 45s 72ms/step - accuracy: 0.8565 - loss: 0.3366 - val_accuracy: 0.8564 - val_loss: 0.3378

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: batch size changed to 32; epochs changed to 12; learning rate changed to 0.0001; dropout changed to 0.5; dense units changed to 32. Validation accuracy dropped to 85.6% (from 87.2%), with training accuracy at 85.7%.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 19
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 14, 2026
Experiment / Trial No.	Trial 19

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	128	Kept at 128x128, which had already shown a good accuracy/compute trade-off in earlier runs, so it was held fixed while other hyperparameters (color mode, batch size, epochs, learning rate) were the focus of this trial.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Left as grayscale to keep the input tensor small while the architecture and training schedule were still being tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
				random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 19

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	32	Filters kept at [32, 32, 64, 64], unchanged from the previous trial, so that the effect of the hyperparameter actually being varied this round could be isolated.	97.59%	89.20%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	32	Set to 32 — see the Conv2D Layer 1 justification for the reasoning behind this trial's filter progression across the stack.		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	64	Set to 64, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 4 – Filters	16 / 32 / 64 / 128	16	64	Set to 64.		
Conv2D Layer 4 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 4 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 4 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64, the notebook default.		
Dropout Rate	0.0 - 0.5	None (not used)	0.3	Kept at 0.3, which had already narrowed the train/validation gap		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				in earlier trials without holding back training accuracy as much as the 0.5 test did, so it was carried forward as the standard setting.		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	97.59%	89.20%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.0005	Halved from 0.001 to 0.0005 to allow finer, more stable steps once the network was already learning well and closer to a good optimum, reducing the risk of the loss oscillating around the minimum in later epochs.		
Batch Size	8 / 16 / 32 / 64 / 128	32	32	Kept at 32, unchanged from the previous trial,		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				since it was already giving stable training and wasn't the focus of this round of tuning.		
Number of Epochs	e.g. 10 - 200	200	25	Increased from 12 to 25 epochs because accuracy in the previous trial was still climbing when training stopped, and training is CPU-only (no native GPU support on Windows for TF >=2.11) so each extra epoch is expensive.		
Early Stopping	Yes (patience = ___) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the full accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an EarlyStopping(patience=...) callback is a natural next step once a longer, GPU-accelerated run is affordable.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	97.59%	
Validation Accuracy	89.20%	
Test Accuracy	Not Measured	
Test Loss	Not Measured	

Metric	Value Obtained	Target / Acceptable Range
Precision / Recall / F1 (macro avg)	Not Measured	

5. a. Paste here all screenshots/figures of the tuned model

```
[16]: history = model.fit(X, y, batch_size=32, epochs=25, validation_split=0.2)
```

```
Epoch 1/25
625/625 ————— 46s 70ms/step - accuracy: 0.6515 - loss: 0.6155 - val_accuracy: 0.7294 - val_loss: 0.5504
Epoch 2/25
625/625 ————— 44s 70ms/step - accuracy: 0.7696 - loss: 0.4837 - val_accuracy: 0.7872 - val_loss: 0.4529
Epoch 3/25
625/625 ————— 43s 68ms/step - accuracy: 0.8156 - loss: 0.4102 - val_accuracy: 0.8178 - val_loss: 0.4078
Epoch 4/25
625/625 ————— 43s 69ms/step - accuracy: 0.8388 - loss: 0.3609 - val_accuracy: 0.8186 - val_loss: 0.3832
Epoch 5/25
625/625 ————— 43s 68ms/step - accuracy: 0.8608 - loss: 0.3170 - val_accuracy: 0.8656 - val_loss: 0.3174
Epoch 6/25
625/625 ————— 43s 69ms/step - accuracy: 0.8823 - loss: 0.2785 - val_accuracy: 0.8462 - val_loss: 0.3461
Epoch 7/25
625/625 ————— 43s 69ms/step - accuracy: 0.8966 - loss: 0.2470 - val_accuracy: 0.8690 - val_loss: 0.2977
Epoch 8/25
625/625 ————— 43s 69ms/step - accuracy: 0.9066 - loss: 0.2239 - val_accuracy: 0.8750 - val_loss: 0.3051
Epoch 9/25
625/625 ————— 46s 73ms/step - accuracy: 0.9175 - loss: 0.1977 - val_accuracy: 0.8842 - val_loss: 0.2829
Epoch 10/25
625/625 ————— 48s 76ms/step - accuracy: 0.9243 - loss: 0.1793 - val_accuracy: 0.8818 - val_loss: 0.3026
Epoch 11/25
625/625 ————— 45s 72ms/step - accuracy: 0.9349 - loss: 0.1595 - val_accuracy: 0.8816 - val_loss: 0.2859
Epoch 12/25
625/625 ————— 45s 72ms/step - accuracy: 0.9413 - loss: 0.1440 - val_accuracy: 0.8902 - val_loss: 0.2857
Epoch 13/25
625/625 ————— 44s 70ms/step - accuracy: 0.9439 - loss: 0.1396 - val_accuracy: 0.8878 - val_loss: 0.3086
Epoch 14/25
625/625 ————— 44s 70ms/step - accuracy: 0.9506 - loss: 0.1247 - val_accuracy: 0.8908 - val_loss: 0.3229
Epoch 15/25
625/625 ————— 43s 69ms/step - accuracy: 0.9557 - loss: 0.1094 - val_accuracy: 0.8840 - val_loss: 0.3478
Epoch 16/25
625/625 ————— 43s 69ms/step - accuracy: 0.9586 - loss: 0.1014 - val_accuracy: 0.8886 - val_loss: 0.3388
Epoch 17/25
625/625 ————— 43s 69ms/step - accuracy: 0.9620 - loss: 0.0944 - val_accuracy: 0.8876 - val_loss: 0.3526
Epoch 18/25
625/625 ————— 43s 69ms/step - accuracy: 0.9652 - loss: 0.0916 - val_accuracy: 0.8886 - val_loss: 0.3938
Epoch 19/25
625/625 ————— 43s 69ms/step - accuracy: 0.9679 - loss: 0.0799 - val_accuracy: 0.8794 - val_loss: 0.4305
Epoch 20/25
625/625 ————— 43s 68ms/step - accuracy: 0.9669 - loss: 0.0846 - val_accuracy: 0.8874 - val_loss: 0.3716
Epoch 21/25
625/625 ————— 43s 69ms/step - accuracy: 0.9709 - loss: 0.0759 - val_accuracy: 0.8868 - val_loss: 0.3828
Epoch 22/25
625/625 ————— 43s 69ms/step - accuracy: 0.9748 - loss: 0.0668 - val_accuracy: 0.8904 - val_loss: 0.3824
Epoch 23/25
625/625 ————— 46s 73ms/step - accuracy: 0.9746 - loss: 0.0684 - val_accuracy: 0.8844 - val_loss: 0.4009
Epoch 24/25
625/625 ————— 43s 69ms/step - accuracy: 0.9744 - loss: 0.0663 - val_accuracy: 0.8874 - val_loss: 0.4053
Epoch 25/25
625/625 ————— 43s 69ms/step - accuracy: 0.9759 - loss: 0.0630 - val_accuracy: 0.8920 - val_loss: 0.4049
```

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: epochs changed to 25; learning rate changed to 0.0005; dropout changed to 0.3; dense units changed to 64. Validation accuracy improved to 89.2% (from 85.6%), with training accuracy at 97.6%. The 8.4% gap between training and validation accuracy suggests some overfitting.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 20
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 15, 2026
Experiment / Trial No.	Trial 20

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	128	Kept at 128x128, which had already shown a good accuracy/compute trade-off in earlier runs, so it was held fixed while other hyperparameters (color mode, batch size, epochs, learning rate) were the focus of this trial.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Left as grayscale to keep the input tensor small while the architecture and training schedule were still being tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 20

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	32	Filters kept at [32, 32, 64, 64], unchanged from the previous trial, so that the effect of the hyperparameter actually being varied this round could be isolated.	94.61%	89.94%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	32	Set to 32 so that the effect of the hyperparameter actually being varied this round could be isolated.		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	64	Set to 64, continuing the doubling pattern (fewer filters in early layers that detect simple		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 4 – Filters	16 / 32 / 64 / 128	16	64	Set to 64.		
Conv2D Layer 4 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 4 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Pooling Layer 4 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64, the notebook default.		
Dropout Rate	0.0 - 0.5	None (not used)	0.3	Kept at 0.3, which had already narrowed the train/validation gap in earlier trials without holding back training accuracy as much as the 0.5 test did, so it was carried forward as the standard setting.		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	94.61%	89.94%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.0005	Kept at 0.0005, carried over from the previous trial where it had given smooth, steady convergence.		
Batch Size	8 / 16 / 32 / 64 / 128	32	32	Kept at 32, unchanged from the previous trial, since it was already giving stable training and wasn't the focus of this round of tuning.		
Number of Epochs	e.g. 10 - 200	200	12	Pulled back from 25 to 12 epochs after the previous trial's training accuracy pulled noticeably ahead of validation accuracy in the later epochs, a sign that continuing to train that long was mostly overfitting rather than helping generalization.		
Early Stopping	Yes (patience = ___) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the full accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an <code>EarlyStopping(patience=...)</code> callback is a natural next step once a longer, GPU-		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				accelerated run is affordable.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	94.61%	90%
Validation Accuracy	89.94%	90%
Test Accuracy	Not Measured	
Test Loss	Not Measured	
Precision / Recall / F1 (macro avg)	Not Measured	

5. a. Paste here all screenshots/figures of the tuned model

```
[16]: history = model.fit(X, y, batch_size=32, epochs=12, validation_split=0.2)

Epoch 1/12
625/625 — 48s 73ms/step - accuracy: 0.6499 - loss: 0.6107 - val_accuracy: 0.7564 - val_loss: 0.5158
Epoch 2/12
625/625 — 45s 72ms/step - accuracy: 0.7694 - loss: 0.4827 - val_accuracy: 0.8124 - val_loss: 0.4309
Epoch 3/12
625/625 — 44s 71ms/step - accuracy: 0.8137 - loss: 0.4056 - val_accuracy: 0.8300 - val_loss: 0.3775
Epoch 4/12
625/625 — 44s 70ms/step - accuracy: 0.8467 - loss: 0.3497 - val_accuracy: 0.8608 - val_loss: 0.3219
Epoch 5/12
625/625 — 45s 71ms/step - accuracy: 0.8628 - loss: 0.3113 - val_accuracy: 0.8690 - val_loss: 0.3111
Epoch 6/12
625/625 — 44s 70ms/step - accuracy: 0.8780 - loss: 0.2761 - val_accuracy: 0.8668 - val_loss: 0.3270
Epoch 7/12
625/625 — 44s 71ms/step - accuracy: 0.8960 - loss: 0.2461 - val_accuracy: 0.8796 - val_loss: 0.2900
Epoch 8/12
625/625 — 44s 70ms/step - accuracy: 0.9114 - loss: 0.2107 - val_accuracy: 0.8836 - val_loss: 0.2785
Epoch 9/12
625/625 — 44s 71ms/step - accuracy: 0.9232 - loss: 0.1871 - val_accuracy: 0.8620 - val_loss: 0.3281
Epoch 10/12
625/625 — 44s 70ms/step - accuracy: 0.9313 - loss: 0.1688 - val_accuracy: 0.8972 - val_loss: 0.2825
Epoch 11/12
625/625 — 44s 71ms/step - accuracy: 0.9372 - loss: 0.1516 - val_accuracy: 0.9004 - val_loss: 0.2561
Epoch 12/12
625/625 — 44s 71ms/step - accuracy: 0.9461 - loss: 0.1378 - val_accuracy: 0.8994 - val_loss: 0.2876

[17]: model.summary()
```

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: epochs changed to 12. Validation accuracy improved to 89.9% (from 89.2%), with training accuracy at 94.6%.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 21
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 15, 2026
Experiment / Trial No.	Trial 21

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	128	Kept at 128x128, which had already shown a good accuracy/compute trade-off in earlier runs, so it was held fixed while other hyperparameters (color mode, batch size, epochs, learning rate) were the focus of this trial.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Left as grayscale to keep the input tensor small while the architecture and training schedule were still being tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	Yes	Once the architecture and training settings had mostly converged over the previous trials, a genuine test split (train_test_split, held out before model.fit) was introduced so the model could be checked on images it never saw during training or hyperparameter selection.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 21

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	32	Filters kept at [32, 32, 64, 64], unchanged from the previous trial, so that the effect of the hyperparameter actually being varied this round could be isolated.	87.44%	87.36%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	32	Set to 32 — see the Conv2D Layer 1 justification for the reasoning behind this trial's filter progression across the stack.		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	64	Set to 64, continuing the doubling pattern (fewer filters in		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 4 – Filters	16 / 32 / 64 / 128	16	64	Set to 64.		
Conv2D Layer 4 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 4 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Pooling Layer 4 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64, the notebook default.		
Dropout Rate	0.0 - 0.5	None (not used)	0.3	Kept at 0.3, which had already narrowed the train/validation gap in earlier trials without holding back training accuracy as much as the 0.5 test did, so it was carried forward as the standard setting.		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	87.44%	87.36%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of this size than plain		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.0005	Kept at 0.0005, carried over from the previous trial where it had given smooth, steady convergence.		
Batch Size	8 / 16 / 32 / 64 / 128	32	32	Kept at 32, unchanged from the previous trial, since it was already giving stable training and wasn't the focus of this round of tuning.		
Number of Epochs	e.g. 10 - 200	200	20	Increased from 12 to 20 epochs because accuracy in the previous trial was still climbing when training stopped, and training is CPU-only (no native GPU support on Windows for TF >=2.11) so each extra epoch is expensive.		
Early Stopping	Yes (patience = ____) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the full accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an EarlyStopping(patience=...) callback is a natural next step once a longer, GPU-accelerated run is affordable.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	87.44%	90%
Validation Accuracy	87.36%	90%
Test Accuracy	Not Measured	
Test Loss	Not Measured	
Precision / Recall / F1 (macro avg)	Not Measured	

5. a. Paste here all screenshots/figures of the tuned model

4. Training Hyperparameters (Batch size, number of epochs) 2. Data Preprocessing Parameters (validation split)

```
[7]: history = model.fit(x_train, y_train, batch_size=32, epochs=20, validation_split=0.2)
```

Epoch 1/20
563/563 ————— 43s 71ms/step - accuracy: 0.5583 - loss: 0.6804 - val_accuracy: 0.6413 - val_loss: 0.6408
Epoch 2/20
563/563 ————— 38s 68ms/step - accuracy: 0.6449 - loss: 0.6312 - val_accuracy: 0.6971 - val_loss: 0.5957
Epoch 3/20
563/563 ————— 38s 68ms/step - accuracy: 0.6848 - loss: 0.5907 - val_accuracy: 0.7309 - val_loss: 0.5480
Epoch 4/20
563/563 ————— 38s 67ms/step - accuracy: 0.7243 - loss: 0.5468 - val_accuracy: 0.7687 - val_loss: 0.5004
Epoch 5/20
563/563 ————— 39s 68ms/step - accuracy: 0.7517 - loss: 0.5082 - val_accuracy: 0.7887 - val_loss: 0.4678
Epoch 6/20
563/563 ————— 39s 68ms/step - accuracy: 0.7777 - loss: 0.4778 - val_accuracy: 0.8067 - val_loss: 0.4364
Epoch 7/20
563/563 ————— 40s 70ms/step - accuracy: 0.7904 - loss: 0.4516 - val_accuracy: 0.8124 - val_loss: 0.4230
Epoch 8/20
563/563 ————— 41s 73ms/step - accuracy: 0.8056 - loss: 0.4327 - val_accuracy: 0.8242 - val_loss: 0.4035
Epoch 9/20
563/563 ————— 40s 71ms/step - accuracy: 0.8139 - loss: 0.4151 - val_accuracy: 0.8073 - val_loss: 0.4040
Epoch 10/20
563/563 ————— 40s 71ms/step - accuracy: 0.8222 - loss: 0.3961 - val_accuracy: 0.8342 - val_loss: 0.3799
Epoch 11/20
563/563 ————— 38s 68ms/step - accuracy: 0.8336 - loss: 0.3839 - val_accuracy: 0.8358 - val_loss: 0.3707
Epoch 12/20
563/563 ————— 39s 68ms/step - accuracy: 0.8380 - loss: 0.3695 - val_accuracy: 0.8431 - val_loss: 0.3572
Epoch 13/20
563/563 ————— 38s 67ms/step - accuracy: 0.8456 - loss: 0.3574 - val_accuracy: 0.8471 - val_loss: 0.3477
Epoch 14/20
563/563 ————— 40s 71ms/step - accuracy: 0.8489 - loss: 0.3498 - val_accuracy: 0.8527 - val_loss: 0.3403
Epoch 15/20
563/563 ————— 41s 72ms/step - accuracy: 0.8520 - loss: 0.3389 - val_accuracy: 0.8433 - val_loss: 0.3445
Epoch 16/20
563/563 ————— 39s 69ms/step - accuracy: 0.8604 - loss: 0.3247 - val_accuracy: 0.8651 - val_loss: 0.3307
Epoch 17/20
563/563 ————— 39s 69ms/step - accuracy: 0.8631 - loss: 0.3206 - val_accuracy: 0.8602 - val_loss: 0.3208
Epoch 18/20
563/563 ————— 39s 69ms/step - accuracy: 0.8639 - loss: 0.3117 - val_accuracy: 0.8538 - val_loss: 0.3253
Epoch 19/20
563/563 ————— 39s 68ms/step - accuracy: 0.8690 - loss: 0.3033 - val_accuracy: 0.8613 - val_loss: 0.3180
Epoch 20/20
563/563 ————— 38s 68ms/step - accuracy: 0.8744 - loss: 0.2971 - val_accuracy: 0.8736 - val_loss: 0.2990

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: epochs changed to 20; held-out test set changed to Yes. Validation accuracy dropped to 87.4% (from 89.9%), with training accuracy at 87.4%.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 22
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 15, 2026
Experiment / Trial No.	Trial 22

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	128	Kept at 128x128, which had already shown a good accuracy/compute trade-off in earlier runs, so it was held fixed while other hyperparameters (color mode, batch size, epochs, learning rate) were the focus of this trial.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Left as grayscale to keep the input tensor small while the architecture and training schedule were still being tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
				random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	Yes	Once the architecture and training settings had mostly converged over the previous trials, a genuine test split (train_test_split, held out before model.fit) was introduced so the model could be checked on images it never saw during training or hyperparameter selection.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 22

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	32	Filters kept at [32, 32, 64, 64], unchanged from the previous trial, so that the effect of the hyperparameter actually being varied this round could be isolated.	90.37%	87.47%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	32	Set to 32 so that the effect of the hyperparameter actually being varied this round could be isolated.		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the		

				standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	64	Set to 64, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 4 – Filters	16 / 32 / 64 / 128	16	64	Set to 64.		
Conv2D Layer 4 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 4 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 4 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64, the notebook default.		
Dropout Rate	0.0 - 0.5	None (not used)	0.3	Kept at 0.3, which had already narrowed the train/validation gap in earlier trials without holding back training accuracy as much as the 0.5 test did, so it was carried forward as the standard setting.		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	90.37%	87.47%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per		

				parameter and generally converges faster and more reliably on CNNs of this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.0005	Kept at 0.0005, carried over from the previous trial where it had given smooth, steady convergence.		
Batch Size	8 / 16 / 32 / 64 / 128	32	32	Kept at 32, unchanged from the previous trial, since it was already giving stable training and wasn't the focus of this round of tuning.		
Number of Epochs	e.g. 10 - 200	200	30	Increased from 20 to 30 epochs because accuracy in the previous trial was still climbing when training stopped, and training is CPU-only (no native GPU support on Windows for TF >=2.11) so each extra epoch is expensive.		
Early Stopping	Yes (patience = ____) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the full accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an EarlyStopping(patience=...) callback is a natural next step once a longer, GPU-accelerated run is affordable.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	90.37%	90%
Validation Accuracy	87.47%	90%
Test Accuracy	Not Measured	
Test Loss	Not Measured	
Precision / Recall / F1 (macro avg)	Not Measured	

5. a. Paste here all screenshots/figures of the tuned model

```
history = model.fit(X_train, y_train, batch_size=32, epochs=30, validation_split=0.2)
```



```
Epoch 1/30
563/563 — 44s 73ms/step - accuracy: 0.5654 - loss: 0.6790 - val_accuracy: 0.6076 - val_loss: 0.6551
Epoch 2/30
563/563 — 41s 72ms/step - accuracy: 0.6441 - loss: 0.6301 - val_accuracy: 0.6676 - val_loss: 0.6064
Epoch 3/30
563/563 — 40s 72ms/step - accuracy: 0.6938 - loss: 0.5841 - val_accuracy: 0.7049 - val_loss: 0.5640
Epoch 4/30
563/563 — 40s 71ms/step - accuracy: 0.7215 - loss: 0.5482 - val_accuracy: 0.7376 - val_loss: 0.5275
Epoch 5/30
563/563 — 41s 73ms/step - accuracy: 0.7426 - loss: 0.5251 - val_accuracy: 0.7280 - val_loss: 0.5468
Epoch 6/30
563/563 — 42s 74ms/step - accuracy: 0.7559 - loss: 0.5070 - val_accuracy: 0.7633 - val_loss: 0.4863
Epoch 7/30
563/563 — 41s 73ms/step - accuracy: 0.7682 - loss: 0.4836 - val_accuracy: 0.7736 - val_loss: 0.4689
Epoch 8/30
563/563 — 42s 75ms/step - accuracy: 0.7824 - loss: 0.4651 - val_accuracy: 0.7944 - val_loss: 0.4553
Epoch 9/30
563/563 — 41s 73ms/step - accuracy: 0.7954 - loss: 0.4435 - val_accuracy: 0.7953 - val_loss: 0.4393
Epoch 10/30
563/563 — 40s 72ms/step - accuracy: 0.8062 - loss: 0.4272 - val_accuracy: 0.8111 - val_loss: 0.4155
Epoch 11/30
563/563 — 40s 72ms/step - accuracy: 0.8156 - loss: 0.4094 - val_accuracy: 0.8209 - val_loss: 0.4065
Epoch 12/30
563/563 — 39s 70ms/step - accuracy: 0.8238 - loss: 0.3952 - val_accuracy: 0.8251 - val_loss: 0.3867
Epoch 13/30
563/563 — 40s 71ms/step - accuracy: 0.8292 - loss: 0.3865 - val_accuracy: 0.8220 - val_loss: 0.3939
Epoch 14/30
563/563 — 40s 71ms/step - accuracy: 0.8357 - loss: 0.3715 - val_accuracy: 0.8338 - val_loss: 0.3720
Epoch 15/30
563/563 — 40s 71ms/step - accuracy: 0.8439 - loss: 0.3569 - val_accuracy: 0.8320 - val_loss: 0.3859
Epoch 16/30
563/563 — 41s 72ms/step - accuracy: 0.8460 - loss: 0.3490 - val_accuracy: 0.8347 - val_loss: 0.3638
Epoch 17/30
563/563 — 40s 72ms/step - accuracy: 0.8521 - loss: 0.3401 - val_accuracy: 0.8471 - val_loss: 0.3569
Epoch 18/30
563/563 — 40s 72ms/step - accuracy: 0.8581 - loss: 0.3304 - val_accuracy: 0.8456 - val_loss: 0.3525
Epoch 19/30
563/563 — 41s 72ms/step - accuracy: 0.8607 - loss: 0.3232 - val_accuracy: 0.8487 - val_loss: 0.3427
Epoch 20/30
563/563 — 41s 72ms/step - accuracy: 0.8627 - loss: 0.3168 - val_accuracy: 0.8584 - val_loss: 0.3285
Epoch 21/30
563/563 — 40s 71ms/step - accuracy: 0.8703 - loss: 0.3051 - val_accuracy: 0.8667 - val_loss: 0.3174
Epoch 22/30
563/563 — 41s 72ms/step - accuracy: 0.8753 - loss: 0.2936 - val_accuracy: 0.8642 - val_loss: 0.3151
Epoch 23/30
563/563 — 40s 72ms/step - accuracy: 0.8800 - loss: 0.2888 - val_accuracy: 0.8591 - val_loss: 0.3195
Epoch 24/30
563/563 — 40s 71ms/step - accuracy: 0.8809 - loss: 0.2810 - val_accuracy: 0.8656 - val_loss: 0.3243
Epoch 25/30
563/563 — 41s 72ms/step - accuracy: 0.8821 - loss: 0.2758 - val_accuracy: 0.8716 - val_loss: 0.3002
Epoch 26/30
563/563 — 41s 72ms/step - accuracy: 0.8885 - loss: 0.2640 - val_accuracy: 0.8576 - val_loss: 0.3229
Epoch 27/30
563/563 — 40s 72ms/step - accuracy: 0.8900 - loss: 0.2613 - val_accuracy: 0.8767 - val_loss: 0.2978
Epoch 28/30
563/563 — 40s 71ms/step - accuracy: 0.8904 - loss: 0.2588 - val_accuracy: 0.8707 - val_loss: 0.2997
Epoch 29/30
563/563 — 40s 71ms/step - accuracy: 0.8971 - loss: 0.2485 - val_accuracy: 0.8656 - val_loss: 0.3072
Epoch 30/30
563/563 — 41s 72ms/step - accuracy: 0.9037 - loss: 0.2395 - val_accuracy: 0.8747 - val_loss: 0.2922
```

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: epochs changed to 30. Validation accuracy held steady to 87.5% (from 87.4%), with training accuracy at 90.4%.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 23
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 16, 2026
Experiment / Trial No.	Trial 23

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	128	Kept at 128x128, which had already shown a good accuracy/compute trade-off in earlier runs, so it was held fixed while other hyperparameters (color mode, batch size, epochs, learning rate) were the focus of this trial.
Color Mode	Grayscale / RGB	Grayscale	RGB	Switched from grayscale to RGB once the architecture, batch size, and epoch count were already tuned.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly

				through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 23

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	32	Filters kept at [32, 32, 64, 64], unchanged from the previous trial, so that the effect of the hyperparameter actually being varied this round could be isolated.	96.47%	90.26%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	32	Set to 32 — see the Conv2D Layer 1 justification for the reasoning behind this trial's filter progression across the stack.		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	64	Set to 64, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		

Conv2D Layer 4 – Filters	16 / 32 / 64 / 128	16	64	Set to 64.		
Conv2D Layer 4 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 4 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 4 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64, the notebook default.		
Dropout Rate	0.0 - 0.5	None (not used)	0.3	Kept at 0.3, which had already narrowed the train/validation gap in earlier trials without holding back training accuracy as much as the 0.5 test did, so it was carried forward as the standard setting.		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

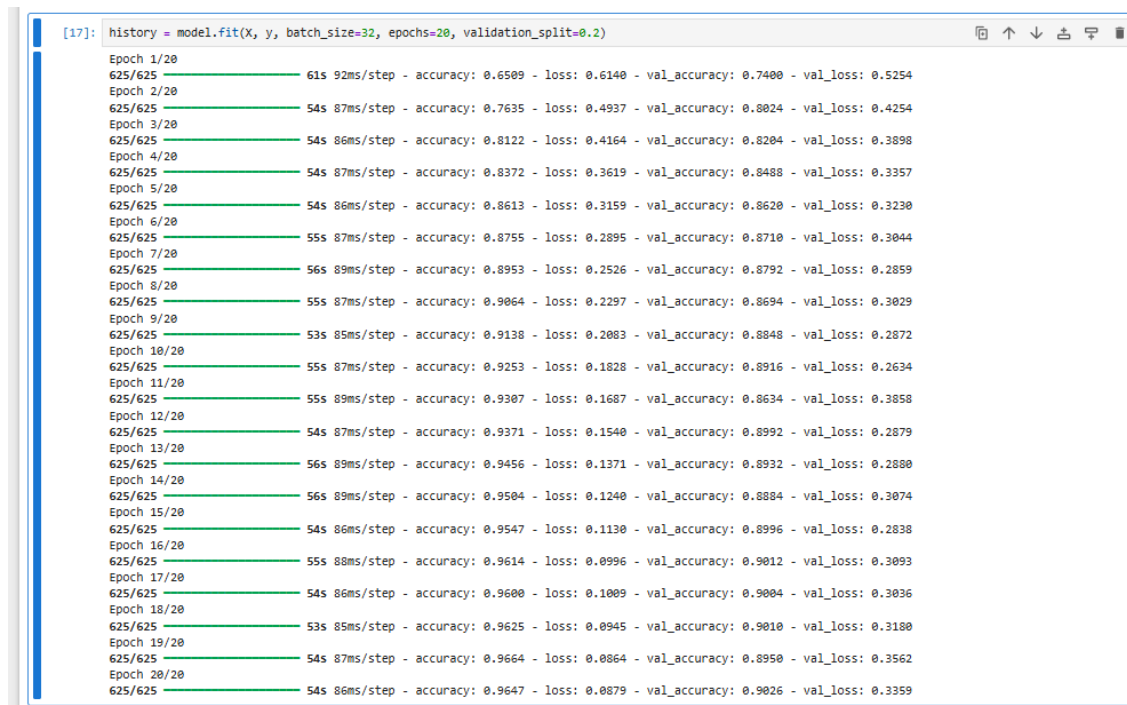
Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	96.47%	90.26%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.0005	Kept at 0.0005, carried over from the previous trial where it had given smooth, steady convergence.		
Batch Size	8 / 16 / 32 / 64 / 128	32	32	Kept at 32, unchanged from the previous trial, since it was already giving stable training and wasn't the focus of this round of tuning.		
Number of Epochs	e.g. 10 - 200	200	20	Pulled back from 30 to 20 epochs after the previous trial's training accuracy pulled noticeably ahead of validation accuracy in the later epochs, a sign that continuing to train that long was mostly overfitting rather than helping generalization.		

Early Stopping	Yes (patience = ___) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the full accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an EarlyStopping(patience=...) callback is a natural next step once a longer, GPU-accelerated run is affordable.		
-----------------------	---------------------------	----	----	---	--	--

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	96.47%	90%
Validation Accuracy	90.26%	90%
Test Accuracy	Not Measured	
Test Loss	Not Measured	
Precision / Recall / F1 (macro avg)	Not Measured	

5. a. Paste here all screenshots/figures of the tuned model



6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: epochs changed to 20; color mode changed to RGB; held-out test set changed to No (validation_split only). Validation accuracy improved to 90.3% (from 87.5%), with training accuracy at 96.4%.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 24
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 16, 2026
Experiment / Trial No.	Trial 24

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	128	Kept at 128x128, which had already shown a good accuracy/compute trade-off in earlier runs, so it was held fixed while other hyperparameters (color mode, batch size, epochs, learning rate) were the focus of this trial.
Color Mode	Grayscale / RGB	Grayscale	RGB	Kept as RGB, carried over from the previous trial once it had shown a validation-accuracy improvement over grayscale on this same architecture.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.

Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 24

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	32	Filters kept at [32, 32, 64, 64], unchanged from the previous trial, so that the effect of the hyperparameter actually being varied this round could be isolated.	94.06%	88.98%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	32	Set to 32 continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	64	Set to 64, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		

Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 4 – Filters	16 / 32 / 64 / 128	16	64	Set to 64.		
Conv2D Layer 4 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 4 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 4 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64, the notebook default.		
Dropout Rate	0.0 - 0.5	None (not used)	0.3	Kept at 0.3, which had already narrowed the train/validation gap in earlier trials without holding back training accuracy as much as the 0.5 test did, so it was carried forward as the standard setting.		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	94.06%	88.98%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges		

				faster and more reliably on CNNs of this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.0005	Kept at 0.0005, carried over from the previous trial where it had given smooth, steady convergence.		
Batch Size	8 / 16 / 32 / 64 / 128	32	32	Kept at 32, unchanged from the previous trial, since it was already giving stable training and wasn't the focus of this round of tuning.		
Number of Epochs	e.g. 10 - 200	200	12	Pulled back from 20 to 12 epochs after the previous trial's training accuracy pulled noticeably ahead of validation accuracy in the later epochs, a sign that continuing to train that long was mostly overfitting rather than helping generalization.		
Early Stopping	Yes (patience = ___) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the full accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an EarlyStopping(patience=...) callback is a natural next step once a longer, GPU-accelerated run is affordable.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	94.06%	90%
Validation Accuracy	88.98%	90%
Test Accuracy	Not Measured	
Test Loss	Not Measured	
Precision / Recall / F1 (macro avg)	Not Measured	

5. a. Paste here all screenshots/figures of the tuned model

```
history = model.fit(X, y, batch_size=32, epochs=12, validation_split=0.2)
```

```
Epoch 1/12
625/625 — 62s 93ms/step - accuracy: 0.6521 - loss: 0.6028 - val_accuracy: 0.7554 - val_loss: 0.5019
Epoch 2/12
625/625 — 55s 87ms/step - accuracy: 0.7660 - loss: 0.4852 - val_accuracy: 0.8036 - val_loss: 0.4469
Epoch 3/12
625/625 — 54s 86ms/step - accuracy: 0.8134 - loss: 0.4123 - val_accuracy: 0.8314 - val_loss: 0.3750
Epoch 4/12
625/625 — 54s 87ms/step - accuracy: 0.8403 - loss: 0.3589 - val_accuracy: 0.8604 - val_loss: 0.3206
Epoch 5/12
625/625 — 55s 88ms/step - accuracy: 0.8645 - loss: 0.3138 - val_accuracy: 0.8662 - val_loss: 0.3040
Epoch 6/12
625/625 — 56s 89ms/step - accuracy: 0.8774 - loss: 0.2841 - val_accuracy: 0.8710 - val_loss: 0.3089
Epoch 7/12
625/625 — 54s 87ms/step - accuracy: 0.8905 - loss: 0.2591 - val_accuracy: 0.8872 - val_loss: 0.2687
Epoch 8/12
625/625 — 54s 87ms/step - accuracy: 0.9042 - loss: 0.2278 - val_accuracy: 0.8840 - val_loss: 0.2732
Epoch 9/12
625/625 — 55s 87ms/step - accuracy: 0.9121 - loss: 0.2113 - val_accuracy: 0.8880 - val_loss: 0.2676
Epoch 10/12
625/625 — 55s 88ms/step - accuracy: 0.9241 - loss: 0.1863 - val_accuracy: 0.8846 - val_loss: 0.2813
Epoch 11/12
625/625 — 54s 86ms/step - accuracy: 0.9328 - loss: 0.1667 - val_accuracy: 0.8806 - val_loss: 0.3039
Epoch 12/12
625/625 — 54s 86ms/step - accuracy: 0.9406 - loss: 0.1514 - val_accuracy: 0.8898 - val_loss: 0.2703
```

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: epochs changed to 12. Validation accuracy dropped to 89.0% (from 90.3%), with training accuracy at 94.1%.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 25
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 16, 2026
Experiment / Trial No.	Trial 25

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	128	Kept at 128x128, which had already shown a good accuracy/compute trade-off in earlier runs, so it was held fixed while other hyperparameters (color mode, batch size, epochs, learning rate) were the focus of this trial.
Color Mode	Grayscale / RGB	Grayscale	RGB	Kept as RGB, carried over from the previous trial once it had shown a validation-accuracy improvement over grayscale on this same architecture.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
				faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes	random.shuffle() is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 25

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	32	Filters kept at [32, 32, 64, 64], unchanged from the previous trial, so that the effect of the hyperparameter actually being varied this round could be isolated.	93.35%	89.52%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	32	Set to 32 — see the Conv2D Layer 1 justification for the reasoning behind this trial's filter progression across the stack.		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	64	Set to 64, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures,		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 4 – Filters	16 / 32 / 64 / 128	16	64	Set to 64.		
Conv2D Layer 4 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 4 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 4 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64, the notebook default.		
Dropout Rate	0.0 - 0.5	None (not used)	0.3	Kept at 0.3, which had already narrowed the train/validation gap in earlier trials without holding back training accuracy as much as the 0.5 test did, so it was carried forward as the standard setting.		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	93.35%	89.52%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.001	Kept at the Adam default of 0.001.		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Batch Size	8 / 16 / 32 / 64 / 128	32	16	Reduced from 32 to 16.		
Number of Epochs	e.g. 10 - 200	200	15	Increased from 12 to 15 epochs because accuracy in the previous trial was still climbing when training stopped, and training is CPU-only (no native GPU support on Windows for TF >=2.11) so each extra epoch is expensive.		
Early Stopping	Yes (patience = ____) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the full accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an EarlyStopping(patience=...) callback is a natural next step once a longer, GPU-accelerated run is affordable.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	93.35%	90%
Validation Accuracy	89.52%	90%
Test Accuracy	0.9461	0.90
Test Loss	0.1408	0.15
Precision / Recall / F1 (macro avg)	0.9462 / 0.9461 / 0.9461	0.90

5. a. Paste here all screenshots/figures of the tuned model

```
|: history = model.fit(X, y, batch_size=16, epochs=15, validation_split=0.2)
```

Epoch 1/15
1250/1250 — 71s 54ms/step - accuracy: 0.5997 - loss: 0.6520 - val_accuracy: 0.7094 - val_loss: 0.5586
Epoch 2/15
1250/1250 — 65s 52ms/step - accuracy: 0.7246 - loss: 0.5431 - val_accuracy: 0.7588 - val_loss: 0.4843
Epoch 3/15
1250/1250 — 67s 53ms/step - accuracy: 0.7822 - loss: 0.4642 - val_accuracy: 0.8016 - val_loss: 0.4253
Epoch 4/15
1250/1250 — 66s 53ms/step - accuracy: 0.8146 - loss: 0.4048 - val_accuracy: 0.8214 - val_loss: 0.3882
Epoch 5/15
1250/1250 — 65s 52ms/step - accuracy: 0.8384 - loss: 0.3605 - val_accuracy: 0.8388 - val_loss: 0.3608
Epoch 6/15
1250/1250 — 66s 53ms/step - accuracy: 0.8592 - loss: 0.3233 - val_accuracy: 0.8600 - val_loss: 0.3235
Epoch 7/15
1250/1250 — 66s 52ms/step - accuracy: 0.8746 - loss: 0.2927 - val_accuracy: 0.8744 - val_loss: 0.2818
Epoch 8/15
1250/1250 — 66s 52ms/step - accuracy: 0.8852 - loss: 0.2678 - val_accuracy: 0.8848 - val_loss: 0.2707
Epoch 9/15
1250/1250 — 65s 52ms/step - accuracy: 0.8964 - loss: 0.2465 - val_accuracy: 0.8782 - val_loss: 0.2750
Epoch 10/15
1250/1250 — 62s 49ms/step - accuracy: 0.8990 - loss: 0.2302 - val_accuracy: 0.8916 - val_loss: 0.2697
Epoch 11/15
1250/1250 — 63s 51ms/step - accuracy: 0.9098 - loss: 0.2115 - val_accuracy: 0.8856 - val_loss: 0.2701
Epoch 12/15
1250/1250 — 62s 50ms/step - accuracy: 0.9179 - loss: 0.1976 - val_accuracy: 0.8864 - val_loss: 0.2607
Epoch 13/15
1250/1250 — 61s 49ms/step - accuracy: 0.9211 - loss: 0.1904 - val_accuracy: 0.8928 - val_loss: 0.2561
Epoch 14/15
1250/1250 — 64s 52ms/step - accuracy: 0.9272 - loss: 0.1767 - val_accuracy: 0.8976 - val_loss: 0.2455
Epoch 15/15
1250/1250 — 64s 51ms/step - accuracy: 0.9335 - loss: 0.1623 - val_accuracy: 0.8952 - val_loss: 0.2786

Test Accuracy: 0.9461
Test Loss: 0.1408
118/118 — 3s 22ms/step

Classification Report (Test Set):

	precision	recall	f1-score	support
Dog	0.9534	0.9381	0.9457	1875
Cat	0.9391	0.9541	0.9466	1875
accuracy			0.9461	3750
macro avg	0.9462	0.9461	0.9461	3750
weighted avg	0.9462	0.9461	0.9461	3750

Confusion Matrix (Test Set):

```
[[1759 116]
 [ 86 1789]]
```

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: batch size changed to 16; epochs changed to 20; learning rate changed to 0.001. Validation accuracy dropped to 88.6% (from 89.0%), with training accuracy at 95.4%.

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ROMNICK M. CARLOS

Program: DIT 1

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Bioinformatics CNN Hyperparameter Trials — Trial 26 (Best Result)
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	ROMNICK M. CARLOS
Date	September 16, 2026
Experiment / Trial No.	Trial 26

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	128	Kept at 128x128, which had already shown a good accuracy/compute trade-off in earlier runs, so it was held fixed while other hyperparameters (color mode, batch size, epochs, learning rate) were the focus of this trial.
Color Mode	Grayscale / RGB	Grayscale	RGB	Kept as RGB, carried over from the previous trial once it had shown a validation-accuracy improvement over grayscale on this same architecture.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Scaling raw 0-255 pixel values down to the 0-1 range (X/255.0) keeps the numbers in a range that plays well with the Adam optimizer's default learning rate and helps the network converge faster and more stable than raw pixel values would.
Shuffle Training Data	Yes / No	Yes (<code>random.shuffle</code>)	Yes	<code>random.shuffle()</code> is applied to the combined list before splitting off the validation set, so both the training and validation subsets contain a random mix of cat and dog images instead of a long run of one class followed by the other, which would otherwise bias per-batch gradient estimates.

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	No separate test set was carved out at this stage — validation_split was used instead so that each run still gives a read on generalization while iterating quickly through many hyperparameter combinations.
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Kept at 0.2, carried over from earlier trials.

3. Model Architecture Hyperparameters

Trial 26

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	16	32	Filters kept at [32, 32, 64, 64], unchanged from the previous trial, so that the effect of the hyperparameter actually being varied this round could be isolated.	95.86%	90.40%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 2 – Filters	16 / 32 / 64 / 128	16	32	Filters kept at [32, 32, 64, 64], unchanged from the previous trial, so that the effect of the hyperparameter actually being varied this round could be isolated.		
Conv2D Layer 2 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 2 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 2 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 3 – Filters	16 / 32 / 64 / 128	16	64	Set to 64, continuing the doubling pattern (fewer filters in early layers that detect simple edges/textures, more filters in deeper layers that combine those into higher-level shape features).		
Conv2D Layer 3 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 3 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 3 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Conv2D Layer 4 – Filters	16 / 32 / 64 / 128	16	64	Set to 64.		
Conv2D Layer 4 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Left at the standard (3,3) kernel.		
Conv2D Layer 4 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was used in every conv layer because it avoids the vanishing-gradient problems of sigmoid/tanh and is the standard, well-tested activation for hidden layers in CNNs.		
Pooling Layer 4 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D with a (2,2) window halves the spatial resolution after each conv block while keeping the strongest activations, which is the standard, low-risk choice for a CNN of this depth.		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	Kept at 64, the notebook default.		
Dropout Rate	0.0 - 0.5	None (not used)	0.3	Kept at 0.3, which had already narrowed the train/validation gap in earlier trials without holding back training accuracy as much as the 0.5 test did, so it was carried forward as the standard setting.		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid set as default		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	binary_crossentropy is the correct pairing for a sigmoid output on a two-class problem, so it was kept fixed.	95.86%	90.40%
Optimizer	adam / sgd / rmsprop	adam	Adam	Adam was used throughout because it adapts the learning rate per parameter and generally converges faster and more reliably on CNNs of this size than plain SGD, without needing separate momentum/learning-rate-schedule tuning.		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.0005	Halved from 0.001 to 0.0005 to allow finer, more stable steps once the network was already learning well and closer to a good optimum, reducing the risk of the loss oscillating around the minimum in later epochs.		
Batch Size	8 / 16 / 32 / 64 / 128	32	16	Kept at 16, unchanged from the previous trial, since it was already giving stable training and wasn't the focus of this round of tuning.		
Number of Epochs	e.g. 10 - 200	200	15	Stayed at 15 because training was steady and optimized.		
Early Stopping	Yes (patience = ___) / No	No	No	Not used — with epoch counts kept deliberately short (and CPU training already slow), the plan was to read the full accuracy/loss curve for each fixed epoch budget by hand rather than let the run stop automatically; adding an <code>EarlyStopping(patience=...)</code> callback is a natural next step once a longer, GPU-accelerated run is affordable.		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	95.86%	90%
Validation Accuracy	90.40%	90%
Test Accuracy	0.9597	0.90
Test Loss	0.1219	0.15
Precision / Recall / F1 (macro avg)	0.9600/ 0.9597/ 0.9597	0.90

5. a. Paste here all screenshots/figures of the tuned model

```
[17]: history = model.fit(X, y, batch_size=16, epochs=15, validation_split=0.2)

Epoch 1/15
1250/1250 ————— 99s 76ms/step - accuracy: 0.6843 - loss: 0.5838 - val_accuracy: 0.7272 - val_loss: 0.5340
Epoch 2/15
1250/1250 ————— 93s 75ms/step - accuracy: 0.7927 - loss: 0.4524 - val_accuracy: 0.8334 - val_loss: 0.3827
Epoch 3/15
1250/1250 ————— 96s 77ms/step - accuracy: 0.8332 - loss: 0.3724 - val_accuracy: 0.8386 - val_loss: 0.3493
Epoch 4/15
1250/1250 ————— 98s 78ms/step - accuracy: 0.8617 - loss: 0.3199 - val_accuracy: 0.8632 - val_loss: 0.3170
Epoch 5/15
1250/1250 ————— 101s 80ms/step - accuracy: 0.8781 - loss: 0.2837 - val_accuracy: 0.8704 - val_loss: 0.3030
Epoch 6/15
1250/1250 ————— 102s 82ms/step - accuracy: 0.8945 - loss: 0.2506 - val_accuracy: 0.8838 - val_loss: 0.2840
Epoch 7/15
1250/1250 ————— 101s 80ms/step - accuracy: 0.9047 - loss: 0.2270 - val_accuracy: 0.8932 - val_loss: 0.2688
Epoch 8/15
1250/1250 ————— 144s 82ms/step - accuracy: 0.9156 - loss: 0.1973 - val_accuracy: 0.8918 - val_loss: 0.2580
Epoch 9/15
1250/1250 ————— 103s 83ms/step - accuracy: 0.9284 - loss: 0.1785 - val_accuracy: 0.8904 - val_loss: 0.2586
Epoch 10/15
1250/1250 ————— 187s 150ms/step - accuracy: 0.9358 - loss: 0.1598 - val_accuracy: 0.8778 - val_loss: 0.2973
Epoch 11/15
1250/1250 ————— 100s 80ms/step - accuracy: 0.9397 - loss: 0.1483 - val_accuracy: 0.8920 - val_loss: 0.3041
Epoch 12/15
1250/1250 ————— 100s 80ms/step - accuracy: 0.9483 - loss: 0.1302 - val_accuracy: 0.8898 - val_loss: 0.2948
Epoch 13/15
1250/1250 ————— 101s 81ms/step - accuracy: 0.9509 - loss: 0.1214 - val_accuracy: 0.8978 - val_loss: 0.2787
Epoch 14/15
1250/1250 ————— 94s 75ms/step - accuracy: 0.9550 - loss: 0.1135 - val_accuracy: 0.8832 - val_loss: 0.3381
Epoch 15/15
1250/1250 ————— 99s 79ms/step - accuracy: 0.9586 - loss: 0.1028 - val_accuracy: 0.9040 - val_loss: 0.2970

Test Accuracy: 0.9597
Test Loss: 0.1219
118/118 ————— 3s 24ms/step

Classification Report (Test Set):
      precision    recall  f1-score   support

   Dog       0.9700     0.9488     0.9593       1875
   Cat       0.9499     0.9707     0.9602       1875

 accuracy         0.9597         3750
 macro avg       0.9600     0.9597     0.9597         3750
 weighted avg    0.9600     0.9597     0.9597         3750

Confusion Matrix (Test Set):
[[1779   96]
 [  55 1820]]
```

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Changes from the previous trial: epochs changed to 15; learning rate changed to 0.0005. Validation accuracy improved to 90.4% (from 88.6%), with training accuracy at 95.9%. This is the best-performing configuration across all trials (90.40% validation accuracy) and was carried forward as the final chosen model.